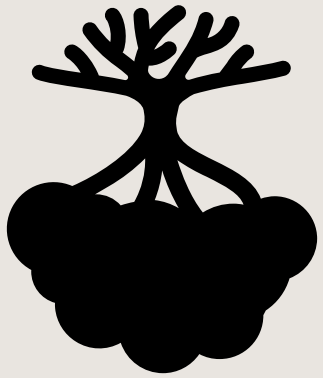
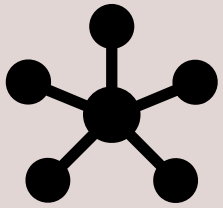


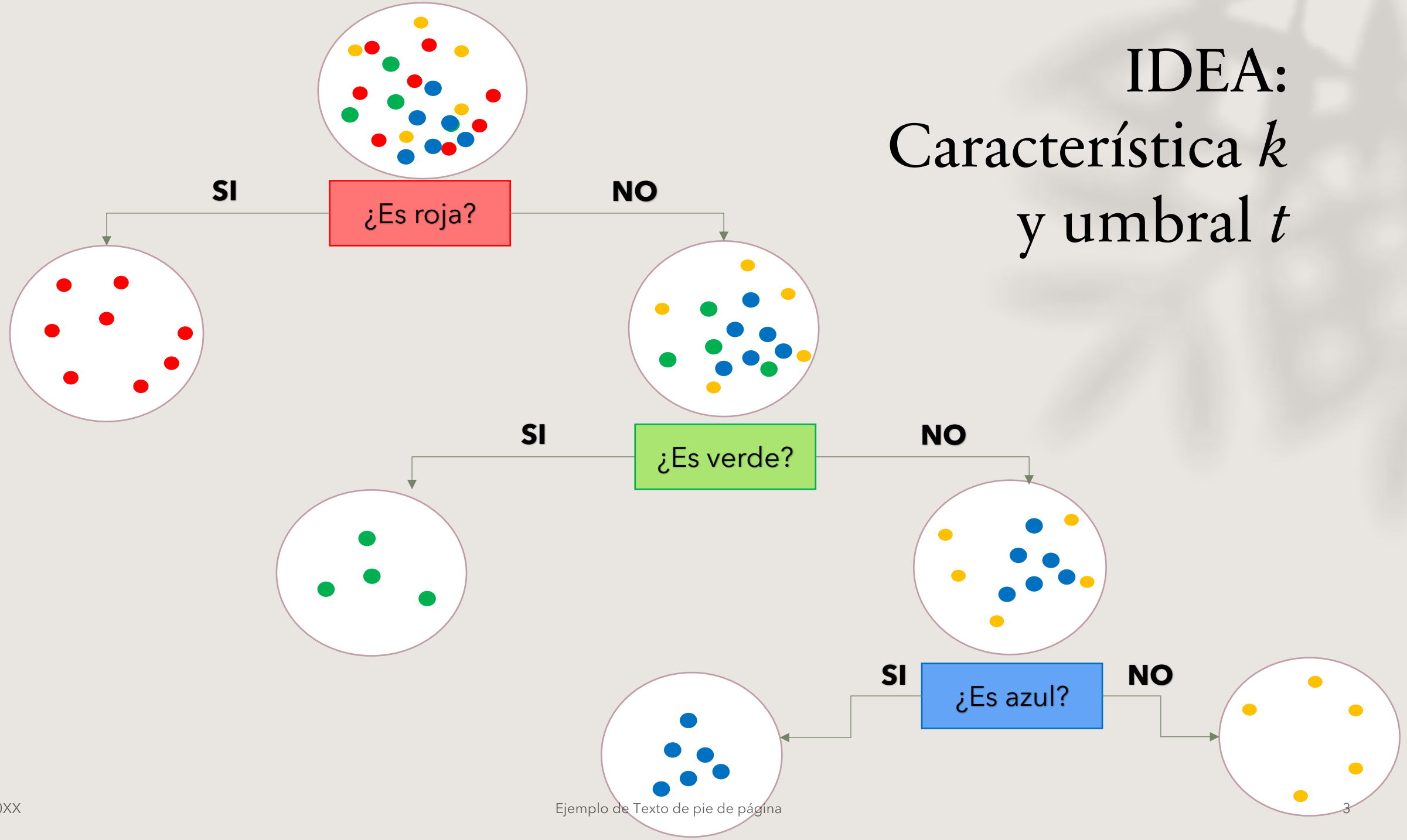
Arboles de decisión

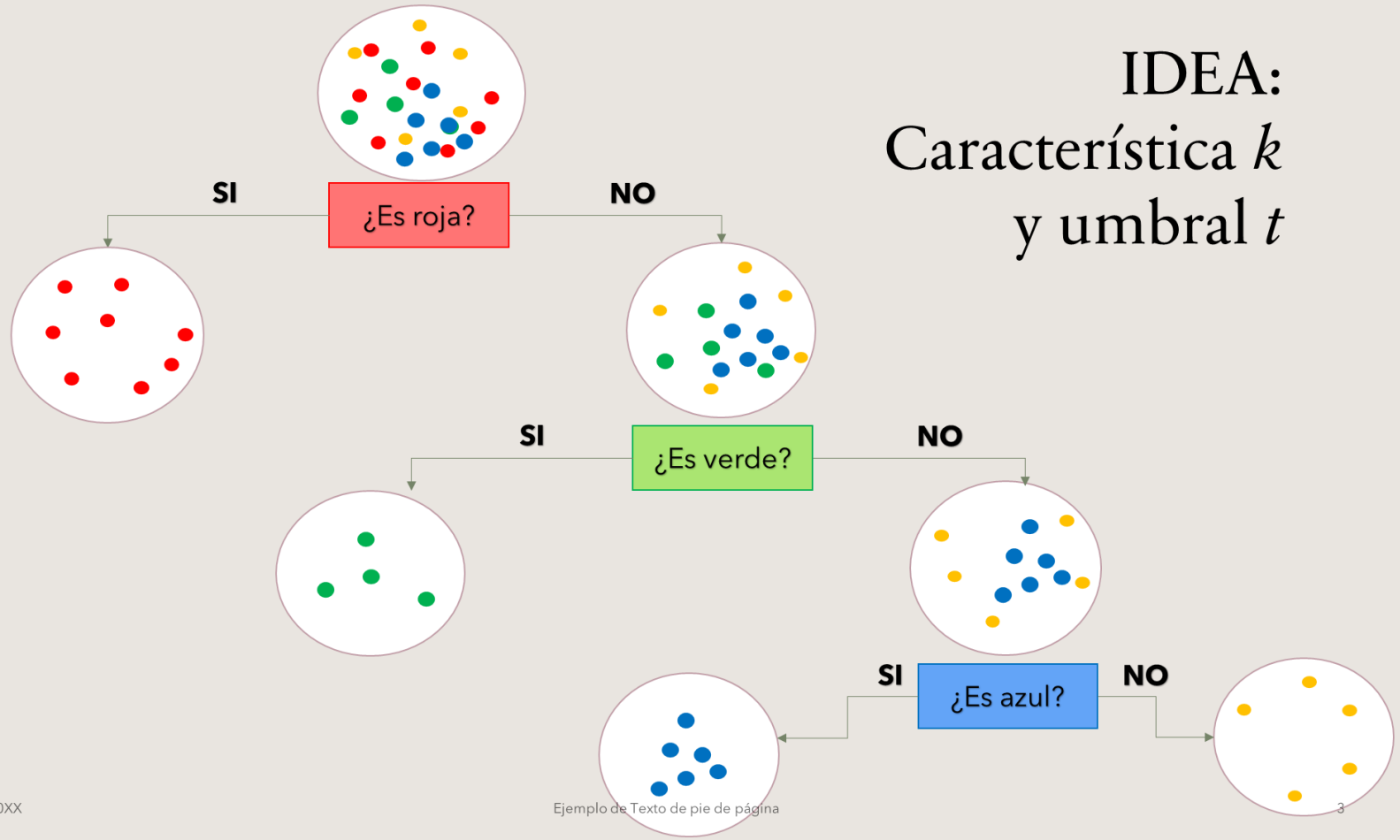
Maria Laura Vranic

Árboles de decisión 	**ELEMENTOS**	
	Nodos de decisión 	Hojas 
	Característica k y umbral t	

Tipos	**Variable objetivo**	**Hojas**
Clasificación	Categórica	Clases
Regresión	Continua	Valores numéricos

IDEA: Característica k y umbral t

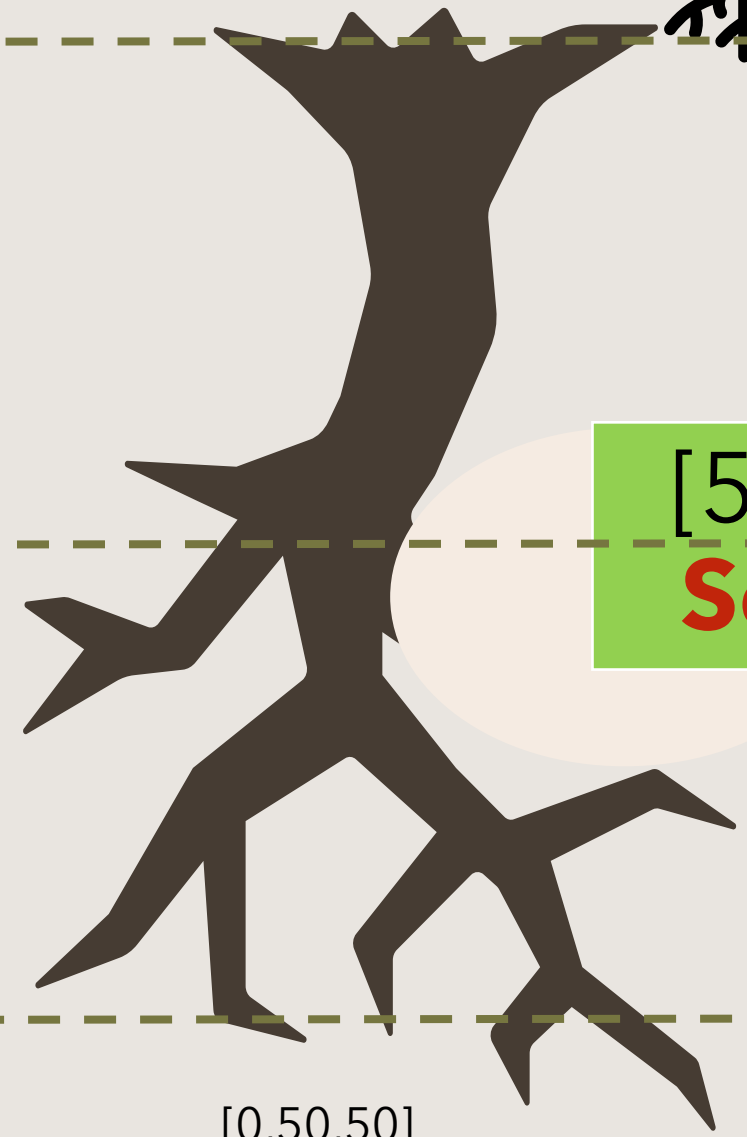




IDEA:
Característica k
y umbral t

La idea es encontrar fronteras de decisión.
Se van generando particiones binarias sobre la región de interés , buscando que cada nueva partición genere un subconjunto de datos lo mas homogéneo posible

OBJETIVO: obtener subconjuntos que pertenezcan en lo posible a una sola categoría



Pethal length (cm)
<=2.45

Raíz

0

TRUE

FALSE

1ra Partición binaria

[50,0,0]
Setosa

Petal width (cm)
<= 1.75 [0,50,50]

Nodo interno

1

2da Partición binaria

[0,49,5]
Versicolor

[0,1,45]
Virginica

2

[0,50,50]
[#Setosa, #Versicolor,#Virginica]

Veamos cómo el árbol realiza predicciones

Pethal length (cm)
<=2.45

TRUE

FALSE

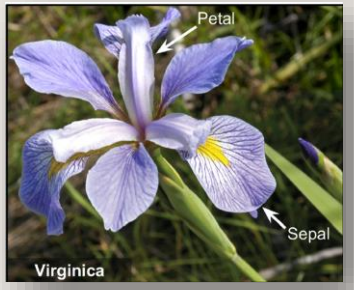


[50,0,0]
Setosa

Petal width (cm)
<= 1.75 [0,50,50]

[0,49,5]
Versicolor

[0,1,45]
Virginica





¿Qué información puede aparecer en un nodo?



- **Condición de división:** Ejemplo: petal length ≤ 2.45 regla que separa los datos en ese nodo.
- **Número de muestras (samples)**: Cantidad de observaciones que llegaron a ese nodo.
- **Distribución de clases (value)**

  Ejemplo: [50, 0, 0] para tres clases (Setosa, Versicolor, Virginica). Muestra cuántos elementos de cada clase hay en el nodo.
- **Clase predicha (class):** La clase mayoritaria en ese nodo.
- **Impureza (Gini o Entropía):** Valor que indica qué tan mezcladas están las clases en el nodo.



¿Cómo se construye automáticamente un árbol de clasificación?

CART

Índice Gini (grado de impureza de un nodo, G_i)

k: $número\ de\ clases$, 3

$$G_i = 1 - \sum_{k=1}^{K} p_{i,k}^2$$

i $número\ de\ nodos$, 2

$p_{i,k}$ *es la proporción de instancias de la clase* k *entre las instancias de entrenamiento en el nodo* j.

Interpretación:

Si todas las observaciones pertenecen a una sola clase → $p_i = 1 \rightarrow Gini = 0$ (**nodo puro**

Si las clases están distribuidas uniformemente → el Gini es máximo (**más impuro**).

El algoritmo de entrenamiento CART

- El algoritmo funciona dividiendo primero el conjunto de entrenamiento en dos subconjuntos mediante una **característica k y un umbral t** (por ejemplo:

  "longitud del pétalo ≤ 2,45 cm").
- ¿Cómo elige k y t? Busca el par (k, t) que produce los subconjuntos más puros (ponderados por su tamaño).
- **Minimiza** la siguiente función de costo

$$J(k, t_k) = \frac{m_{left}}{m} G_{left} + \frac{m_{right}}{m} G_{right}$$

Donde

$G_{\frac{left}{right}}$ mide la impureza del subconjunto izquierdo/derecho

$m_{left/right}$ el número de instancias en el subconjunto izquierdo/derecho.

$$G_i = 1 - \sum_{k=1}^{K} p_{i,k}^2$$

Pethal length (cm) <=2.45

$$G_0 = 1 - \left(\frac{50}{150}\right)^2 - \left(\frac{50}{150}\right)^2 - \left(\frac{50}{150}\right)^2 = 0{,}664$$

TRUE

FALSE

[50,0,0]
Setosa

Petal width (cm) <= 1.75
[0,50,50]

$$G_{1right} = 1 - \left(\frac{50}{50}\right)^2 - \left(\frac{0}{50}\right)^2 - \left(\frac{0}{50}\right)^2 = 0$$

$$G_{1left} = 1 - \left(\frac{0}{100}\right)^2 - \left(\frac{50}{100}\right)^2 - \left(\frac{50}{100}\right)^2 = 0.5$$

[0,49,5]
Versicolor

[0,1,45]
Virginica

$$G_{2left} = 1 - \left(\frac{0}{54}\right)^2 - \left(\frac{49}{54}\right)^2 - \left(\frac{5}{54}\right)^2 = 0{,}168$$

$$G_{2right} = 1 - \left(\frac{0}{46}\right)^2 - \left(\frac{1}{46}\right)^2 - \left(\frac{45}{46}\right)^2 = 0.043$$

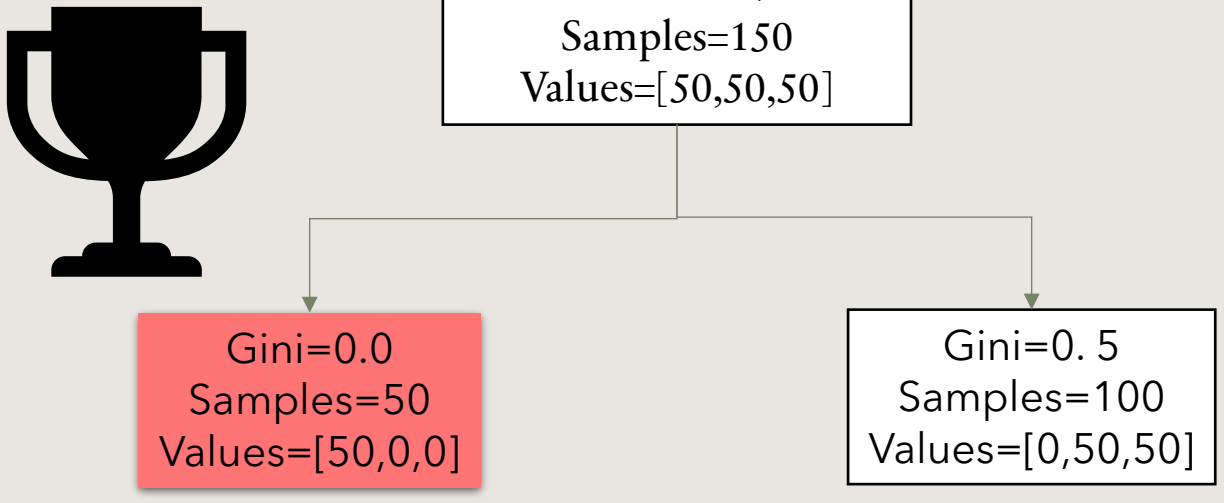
$$J(k, t_k) = \frac{m_{left}}{m} G_{left} + \frac{m_{right}}{m} G_{right}$$

donde $\begin{cases} G_{\frac{left}{right}} \\ m_{left/right} \end{cases}$

mide la impureza del subconjunto izquierdo/derecho

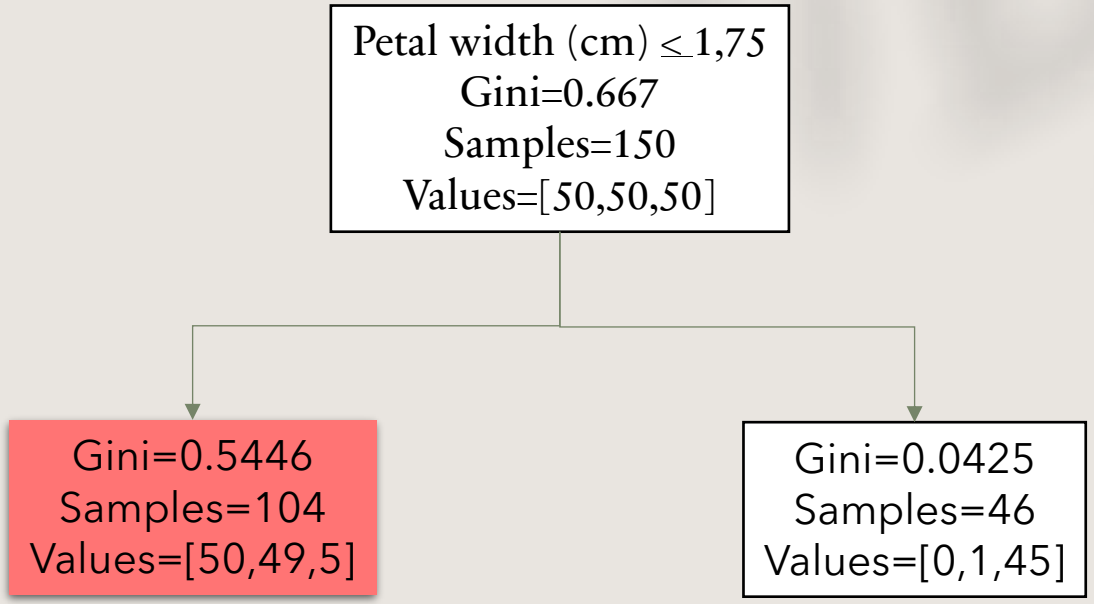
el número de instancias en el subconjunto izquierdo/derecho.

Gráfico A



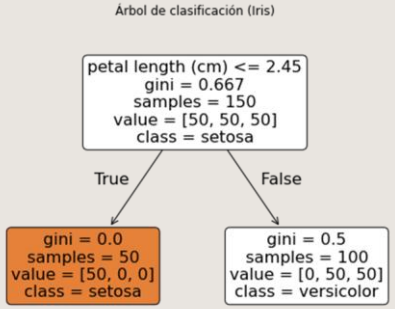
$$J(k = 1, t_1 = 2.45) = \frac{(50 + 0 + 0)}{150} \times 0 + \frac{(0 + 50 + 50)}{150} \times 0.5 = 0{,}333$$

Gráfico B



$$J(k = 1, t_1 = 1.75) = \frac{(50 + 49 + 5)}{150} \times 0{,}5446 + \frac{(0 + 1 + 45)}{150} \times 0{,}0425 = 0{,}3906$$

Algoritmo CART

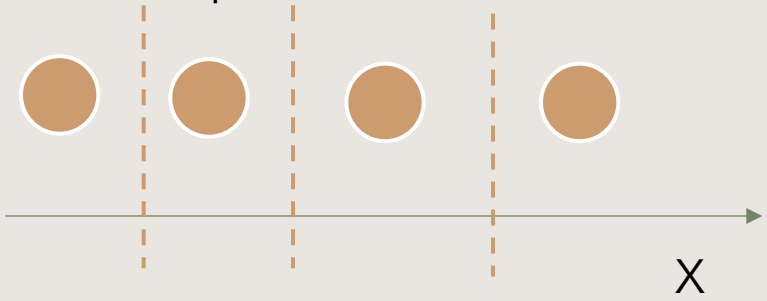


Mínimo $J(k, t_k)$

Que los nodos sean puros (**Gini = 0**), O se **alcance un criterio de parada** (profundidad máxima, mínimo de muestras, etc.).

Calcular todos los posibles umbrales → **Calcular índices Gini y costo** → **Escoger el umbral con menor costo** → **Repetir**

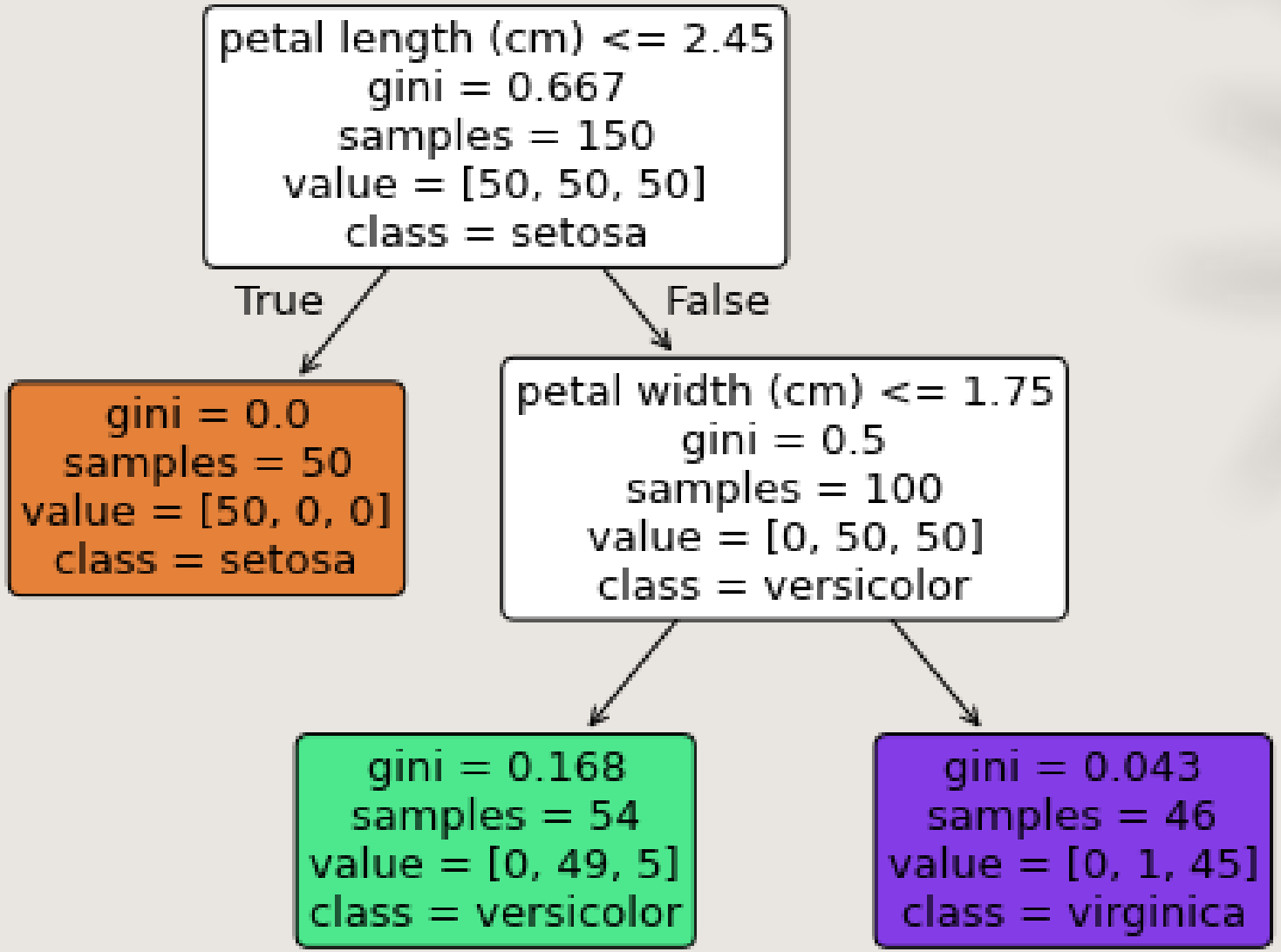
Son n-1 para cada variable



$$G_i = 1 - \sum_{k=1}^{K} p_{i,k}^2$$

$$J(k, t_k) = \frac{m_{left}}{m} G_{left} + \frac{m_{right}}{m} G_{right}$$

Árbol de clasificación (Iris)

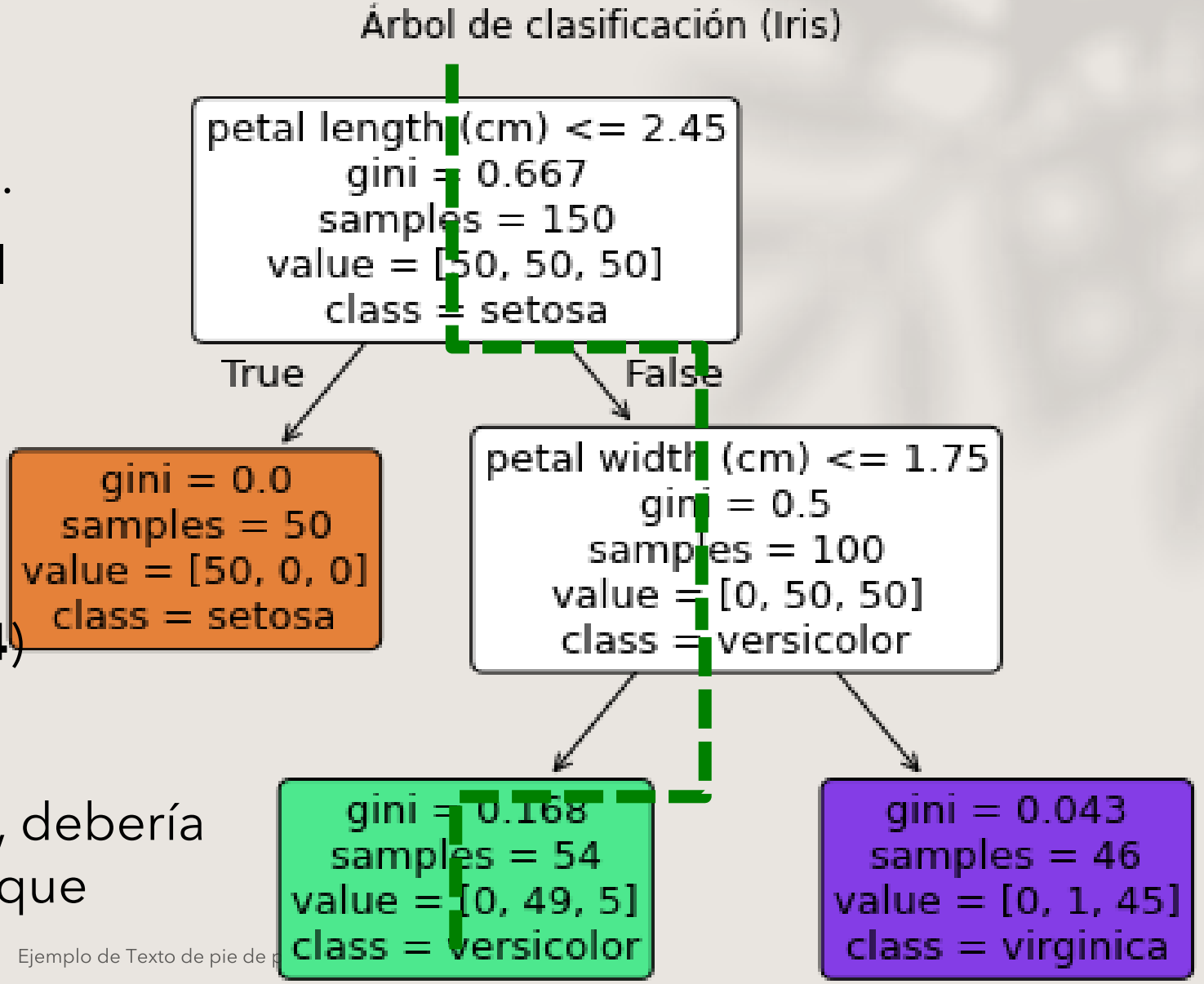


Estimación de probabilidades de clase

Se ha encontrado una flor cuyos pétalos miden:
5 cm de largo y **1,5 cm de ancho**.

El nodo hoja correspondiente es el nodo situado a la izquierda de la profundidad 2, por lo que el árbol de decisión debería generar las siguientes probabilidades:
0 % para Iris **setosa (0/54),**
90,7 % para **Iris versicolor** (**49/54**)
y **9,3 %** para **Iris virginica** (**5/54**).

Y si se le pide que prediga la clase, debería generar Iris versicolor (clase 1) porque tiene la mayor probabilidad.



Estimación de probabilidades de clase

Se ha encontrado una flor cuyos pétalos miden:

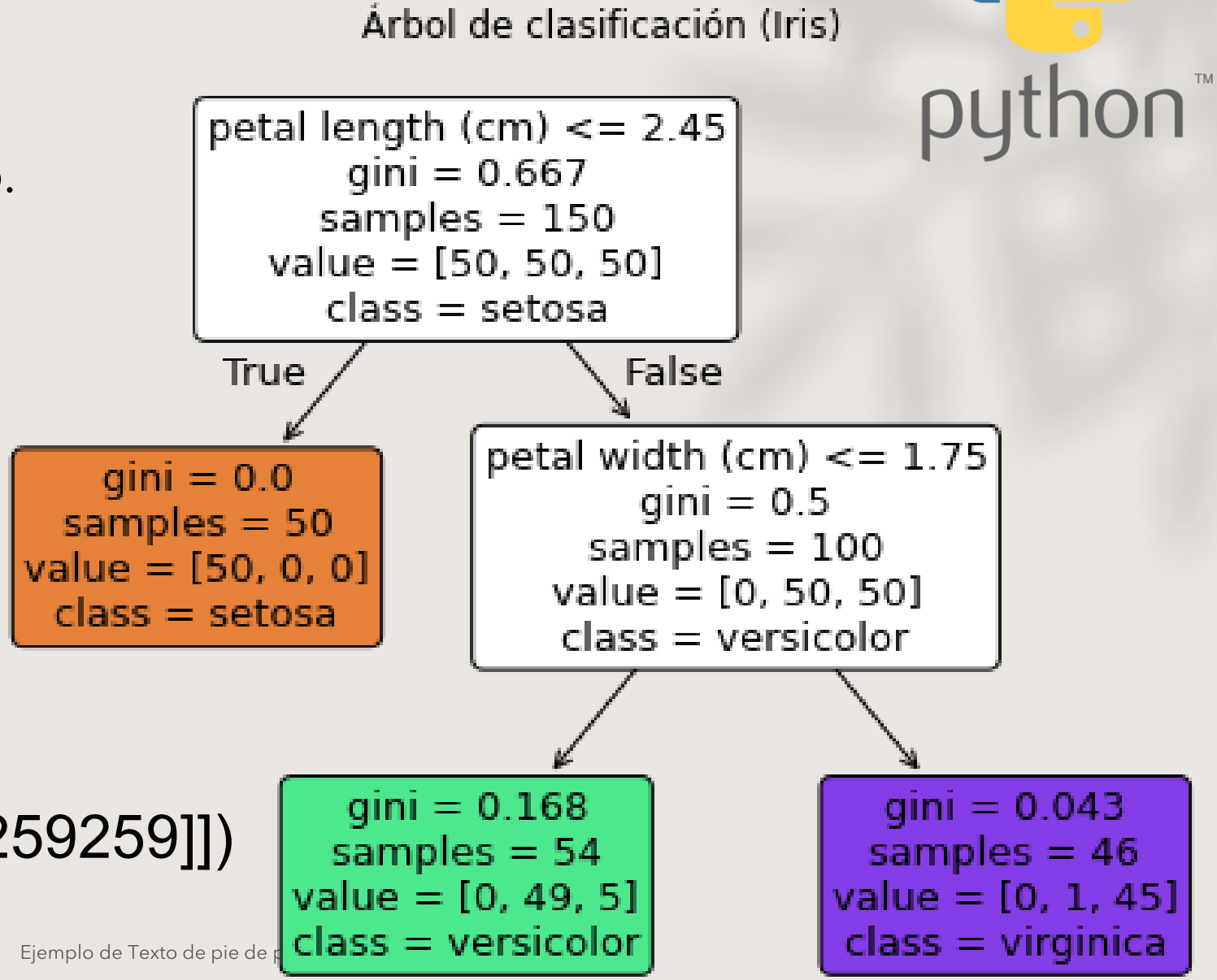
5 cm de largo y **1,5 cm de ancho**.

tree_clf.predict_proba([[5, 1.5]])

Calcula las **probabilidades** de que la muestra [petal length = 5, petal width = 1.5] pertenezca a cada clase.

Resultado:

array([[0. , 0.90740741, 0.09259259]])



Complejidad Computacional

Para hacer una predicción, el árbol se recorre desde la raíz hasta una hoja.

Como los árboles suelen estar balanceados, este recorrido implica pasar por aproximadamente **O(log(m))** nodos, siendo m el número de muestras.

En cada nodo solo se evalúa una característica, por lo que la predicción es rápida y no depende del número de características



¿Qué significa O(log(m))?

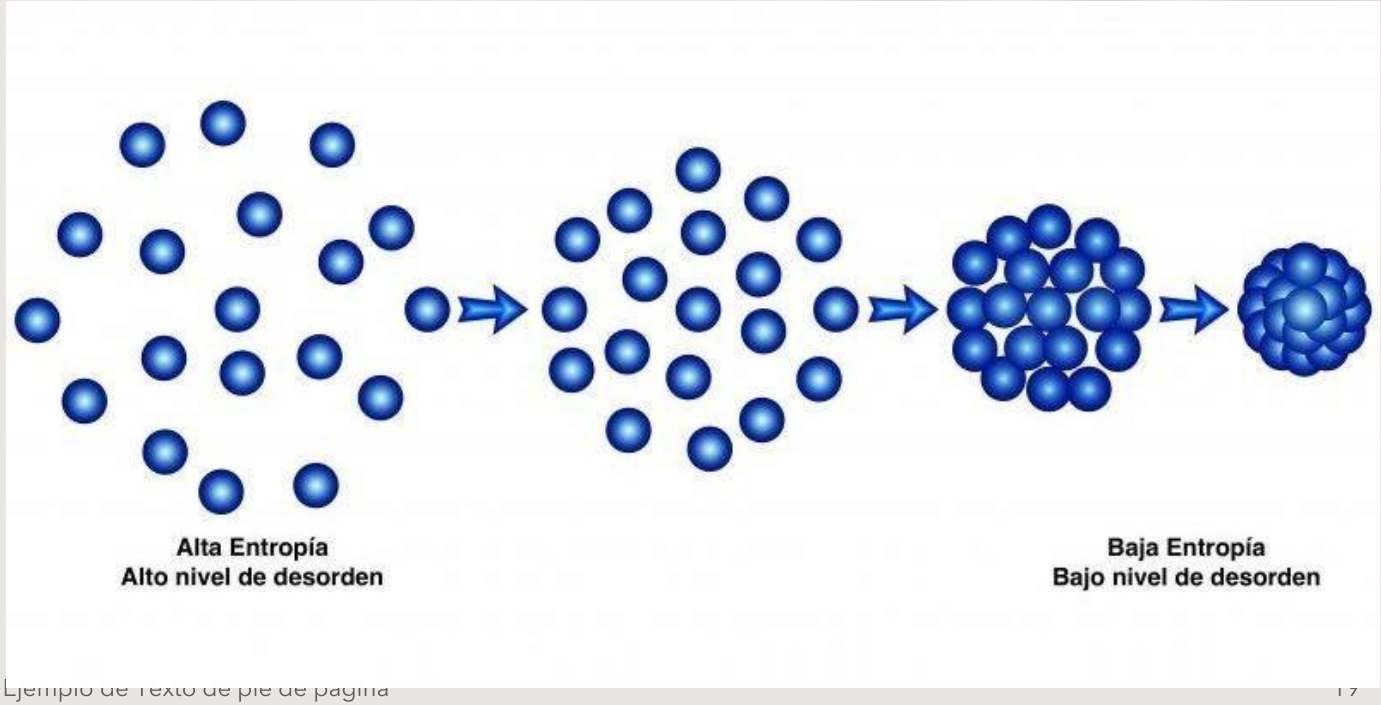
- "**O**" es la notación de "orden de magnitud" o "Big O", que indica el comportamiento del algoritmo en el peor caso.
- "**log(m)**" significa que el tiempo de ejecución crece de forma logarítmica con respecto al número de muestras m.
- 🔍 Ejemplo intuitivo:
- Si tenés:**1.000 muestras** → el algoritmo recorre aproximadamente $\log_2(1000) \approx$ **10 nodos**.
- **1.000.000 muestras** → recorre $\log_2(1.000.000) \approx$ **20 nodos**.
- Aunque el número de muestras crezca mucho, el número de pasos necesarios para hacer una predicción crece muy lentamente.



¿Impureza de Gini o entropía?

- Por defecto, se usa **impureza de Gini** para dividir nodos.
- Se puede usar **entropía** configurando el hiperparámetro criterion="entropy".

$$H_i = -\sum_{\substack{k=1 \\ p_{i,k} \neq 0}}^{n} p_{i,k} \log_2 (p_{i,k})$$

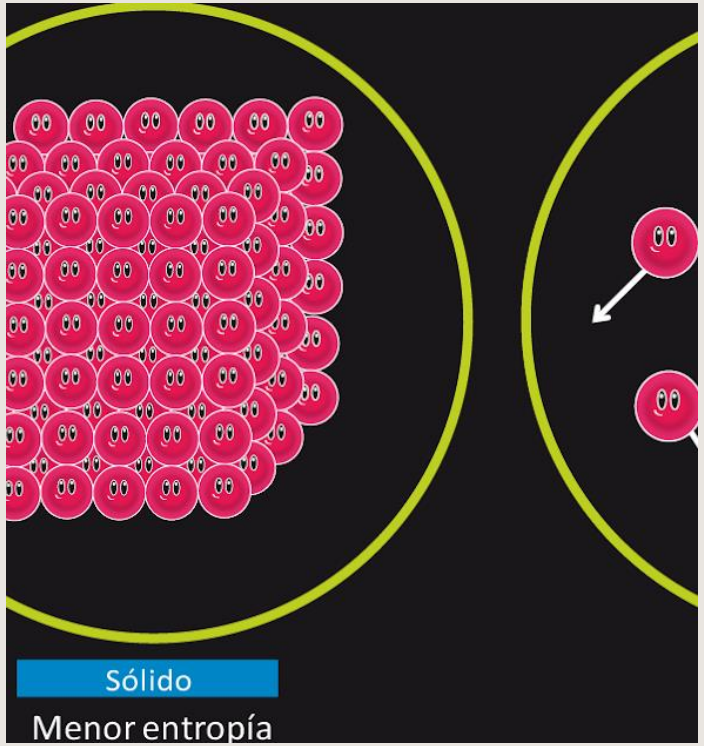


Origen y significado de la entropía

En teoría de la información, mide el contenido informativo promedio de un mensaje.



Proviene de la termodinámica como medida de desorden



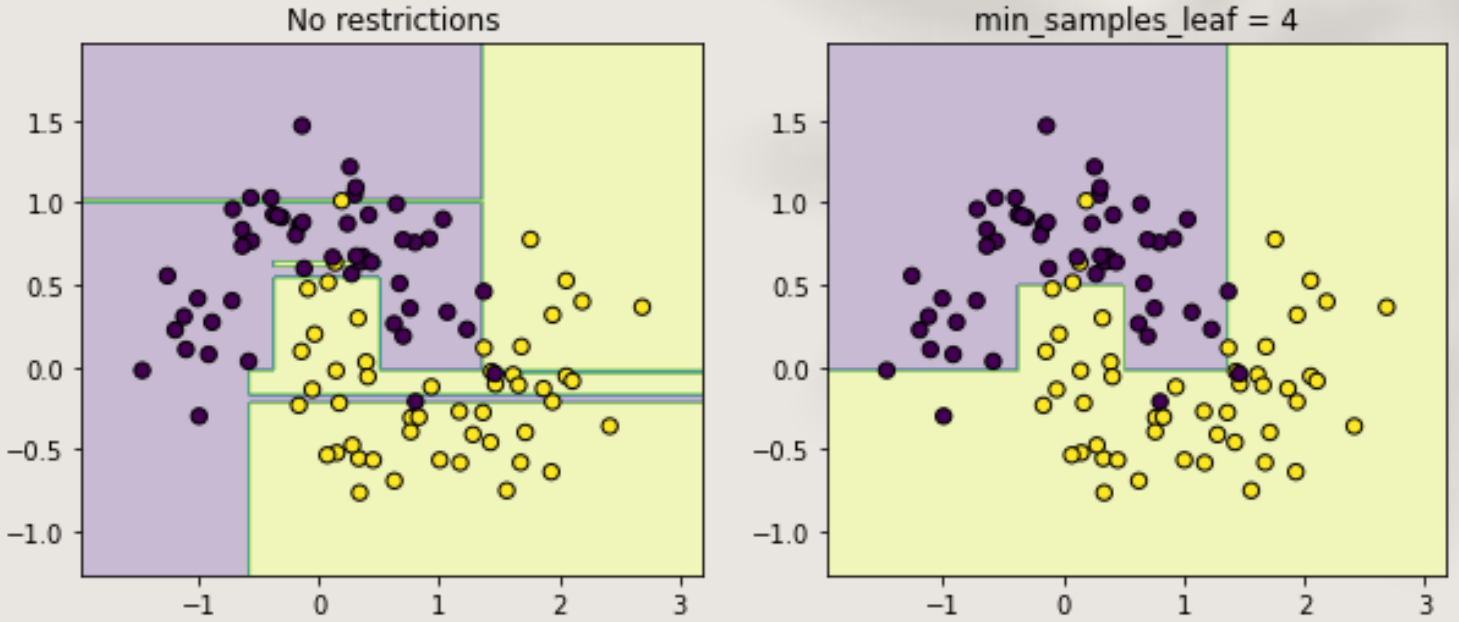
En aprendizaje automático, mide la impureza: entropía = 0 cuando todas las instancias son de una sola clase





Naturaleza del modelo

- Los árboles de decisión son **modelos no paramétricos**: no tienen un número fijo de parámetros antes del entrenamiento, su estructura se adapta a los datos.
- Esto les da mucha flexibilidad, pero también **alto riesgo de sobreajuste** si no se aplican restricciones.



Se definen zonas muy pequeñas, el set de datos se ajusta muy bien a los datos de entrenamiento pero no predice correctamente



Prevenir el sobreajuste



Restringir el número de nodos

Pre poda

Definir el valor de los hiperparámetros:

- Profundidad máxima
- Número mínimo de datos en un nodo
- Número mínimo de datos en una hoja



Post poda

Poda de complejidad de costos, paramétro alfa. **¿Para qué sirve?**
Si α = 0 → no se penaliza la complejidad → árbol muy grande (riesgo de sobreajuste).
Si α aumenta → se penaliza la complejidad → el árbol se poda más → modelo más simple (menos sobreajuste, pero puede perder precisión).

Observando el árbol

Las variables que aparecen **más cerca de la raíz** (arriba) son las más importantes porque se usan primero para dividir los datos.

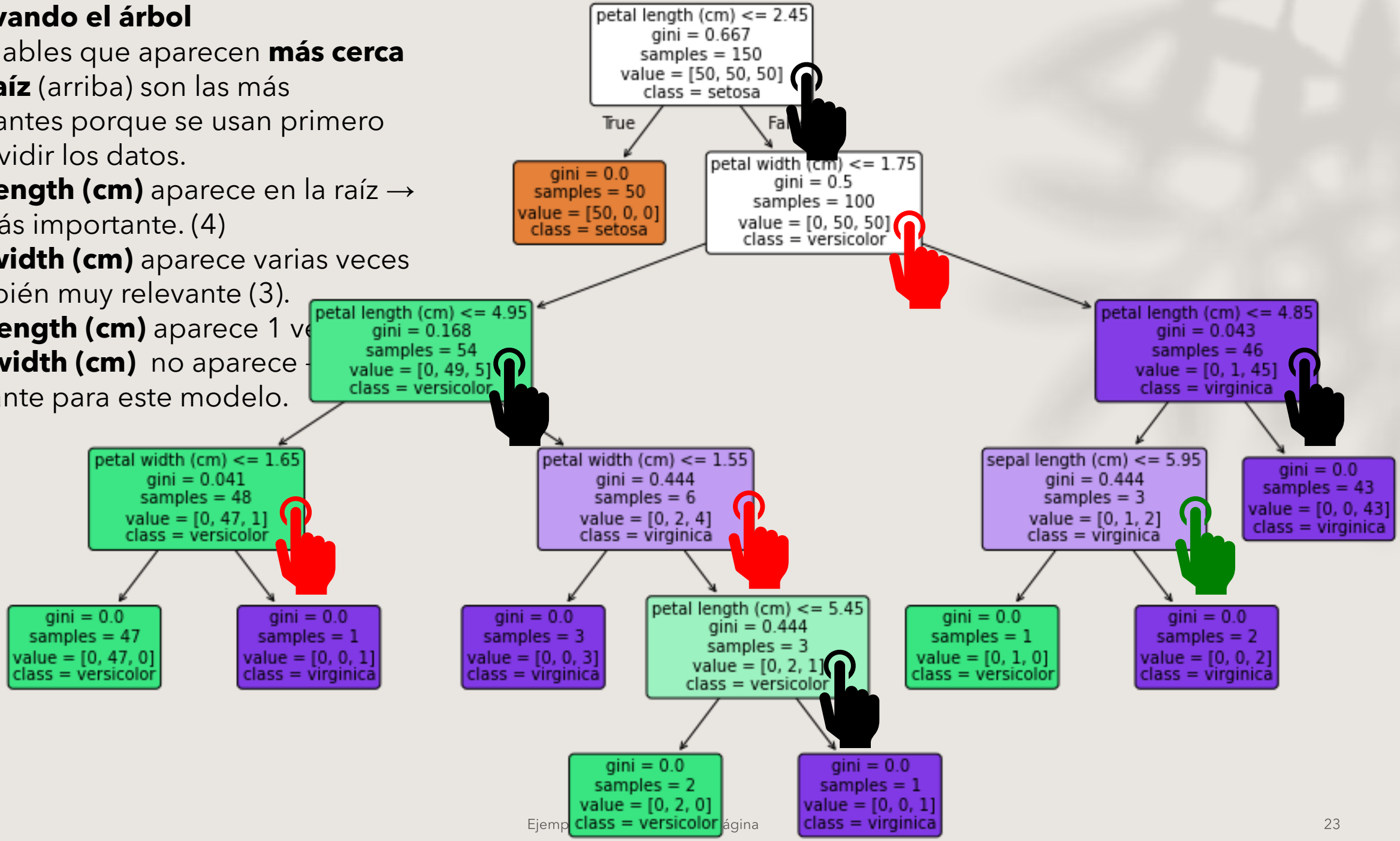
petal length (cm) aparece en la raíz → es la más importante. (4)

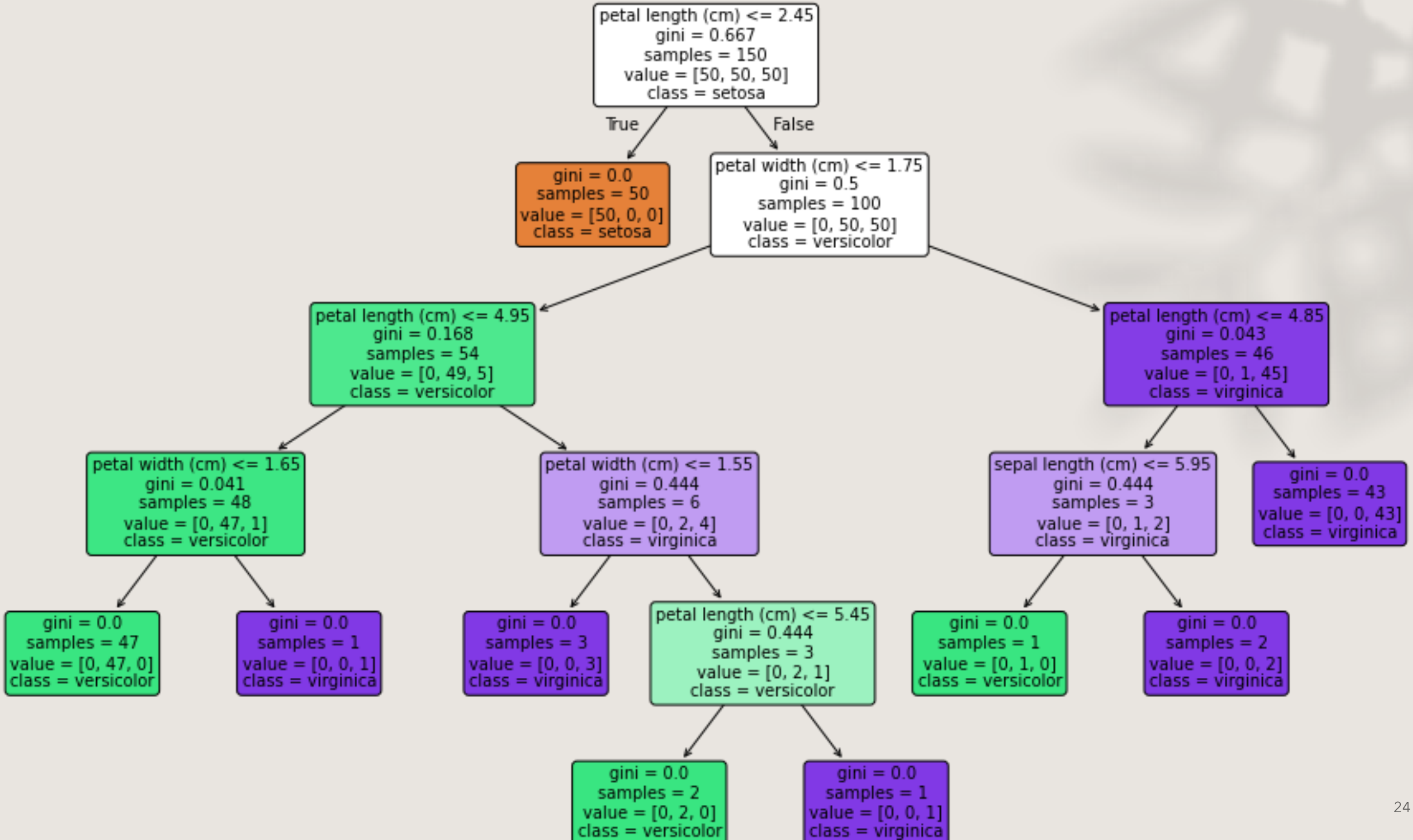
petal width (cm) aparece varias veces → también muy relevante (3).

sepal length (cm) aparece 1 ve

sepal width (cm) no aparece

irrelevante para este modelo.




petal length (cm) <= 2.45
gini = 0.667
samples = 150
value = [50, 50, 50]
class = setosa
True
False
gini = 0.0
samples = 50
value = [50, 0, 0]
class = setosa
petal width (cm) <= 1.75
gini = 0.5
samples = 100
value = [0, 50, 50]
class = versicolor
petal length (cm) <= 4.95
gini = 0.168
samples = 54
value = [0, 49, 5]
class = versicolor
petal length (cm) <= 4.85
gini = 0.043
samples = 46
value = [0, 1, 45]
class = virginica
petal width (cm) <= 1.65
gini = 0.041
samples = 48
value = [0, 47, 1]
class = versicolor
petal width (cm) <= 1.55
gini = 0.444
samples = 6
value = [0, 2, 4]
class = virginica
sepal length (cm) <= 5.95
gini = 0.444
samples = 3
value = [0, 1, 2]
class = virginica
gini = 0.0
samples = 43
value = [0, 0, 43]
class = virginica
gini = 0.0
samples = 47
value = [0, 47, 0]
class = versicolor
gini = 0.0
samples = 1
value = [0, 0, 1]
class = virginica
gini = 0.0
samples = 3
value = [0, 0, 3]
class = virginica
petal length (cm) <= 5.45
gini = 0.444
samples = 3
value = [0, 2, 1]
class = versicolor
gini = 0.0
samples = 1
value = [0, 1, 0]
class = versicolor
gini = 0.0
samples = 2
value = [0, 0, 2]
class = virginica
gini = 0.0
samples = 2
value = [0, 2, 0]
class = versicolor
gini = 0.0
samples = 1
value = [0, 0, 1]
class = virginica

✅ Paso 1: Fórmula

Para cada división:

$$\text{Reducción de impureza} = Gini_{padre} - \left(\frac{n_{izq}}{n_{total}} Gini_{izq} + \frac{n_{der}}{n_{total}} Gini_{der} \right)$$

Luego:

$$\text{Importancia variable} = \sum \left(\frac{n_{nodo}}{n_{total}} \times \text{Reducción} \right)$$

$$\Delta = 0.168 - \left(\frac{48}{54}(0.041) + \frac{6}{54}(0.444) \right)$$

$$= 0.168 - (0.0365 + 0.0493) = 0.168 - 0.0858 = 0.0822$$

Ponderación:

$$\text{Contribución} = \frac{54}{150} \times 0.0822 \approx 0.0296$$

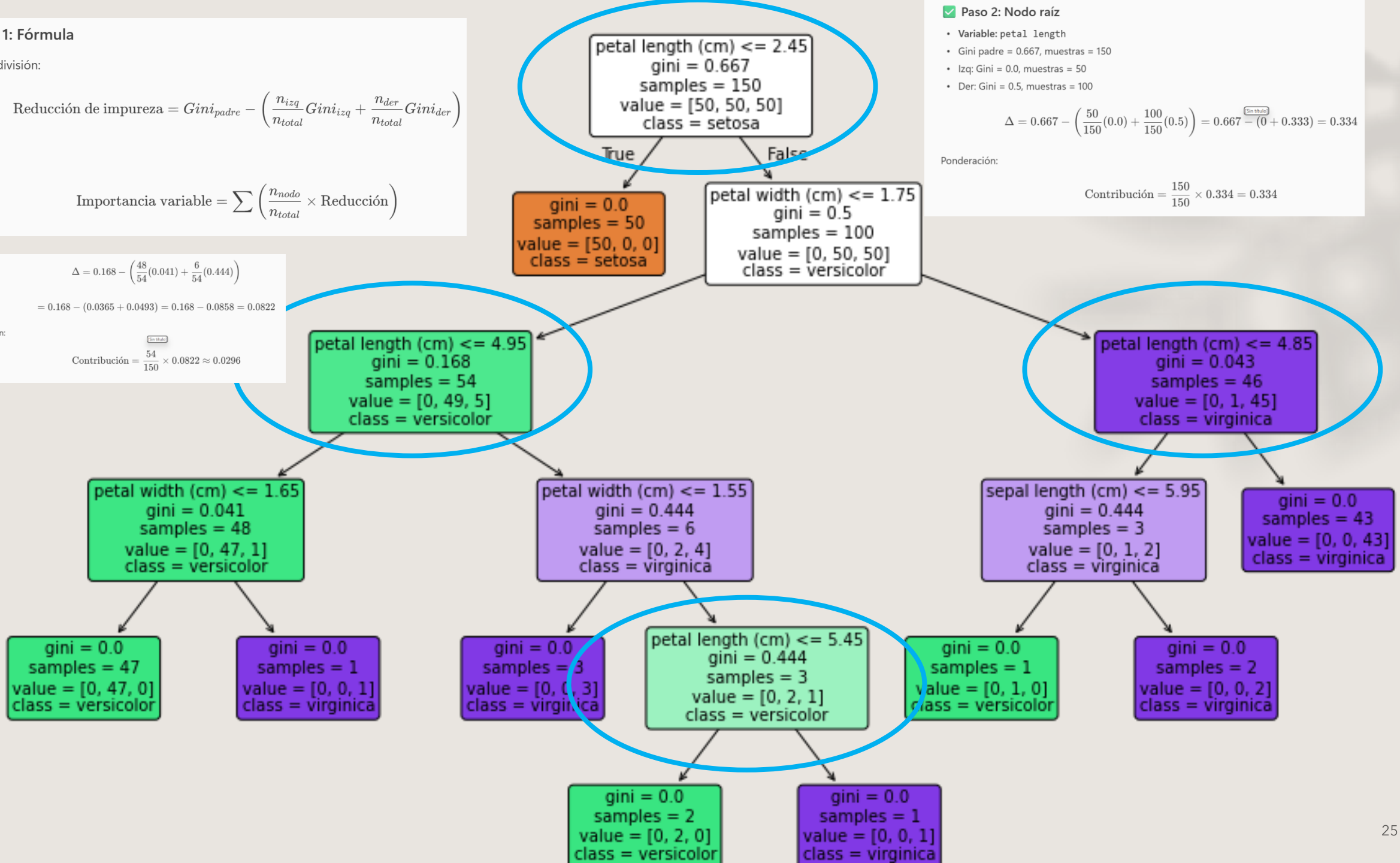
✅ Paso 2: Nodo raíz

- **Variable:** `petal length`
- Gini padre = 0.667, muestras = 150
- Izq: Gini = 0.0, muestras = 50
- Der: Gini = 0.5, muestras = 100

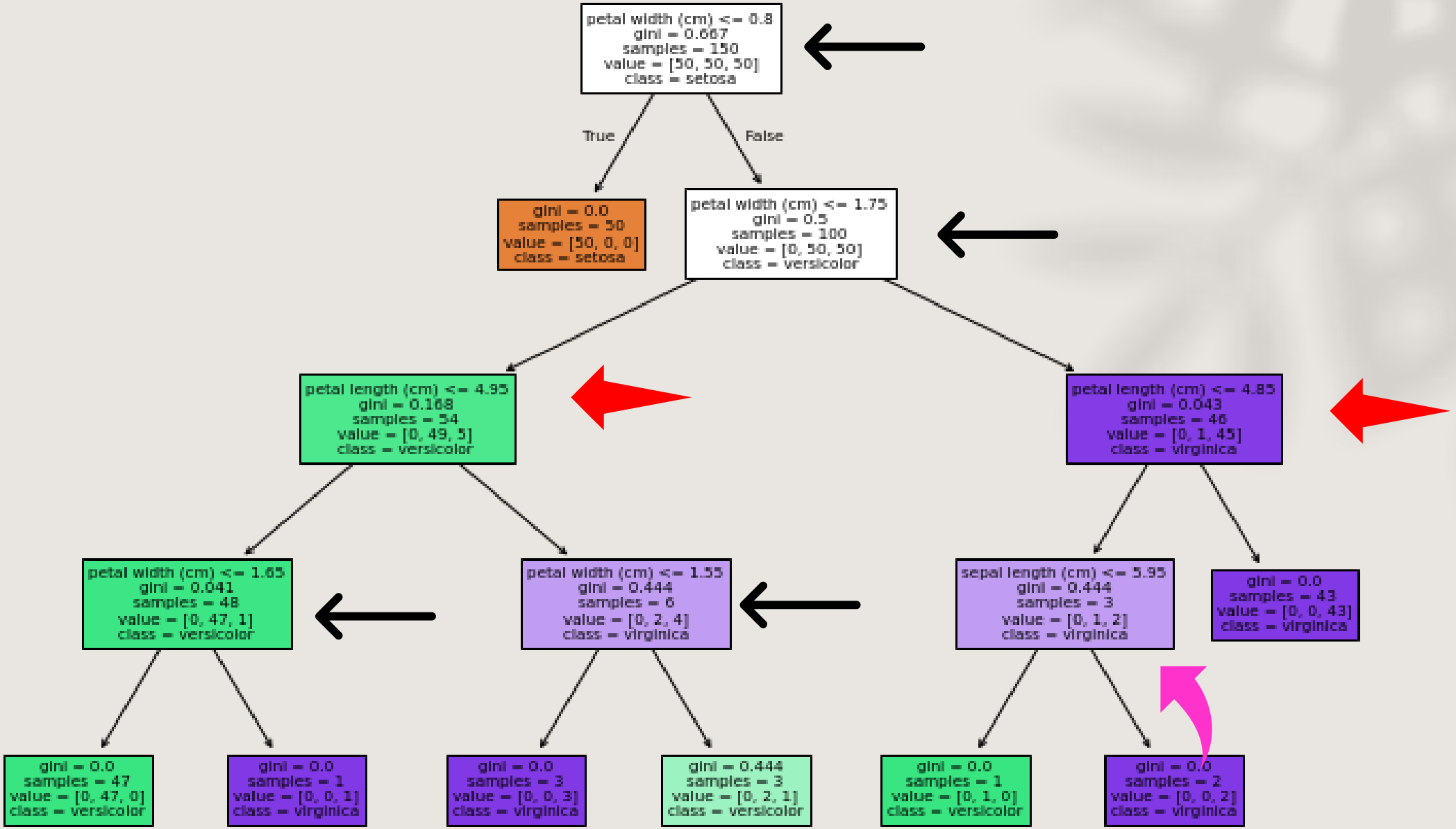
$$\Delta = 0.667 - \left(\frac{50}{150}(0.0) + \frac{100}{150}(0.5) \right) = 0.667 - (0 + 0.333) = 0.334$$

Ponderación:

$$\text{Contribución} = \frac{150}{150} \times 0.334 = 0.334$$



Árbol de Decisión Óptimo (Iris)



```python
import pandas as pd

# Importancia de las variables
importances = tree_clf.feature_importances_
features = iris.feature_names

# Mostrar en tabla ordenada
pd.DataFrame({'Feature': features, 'Importance': importances}).sort_values(by='Importance', ascending=False)
```

[5] ✓ 0.7s

	Feature	Importance
2	petal length (cm)	0.893264
3	petal width (cm)	0.087626
1	sepal width (cm)	0.019110
0	sepal length (cm)	0.000000

Árboles de regresión



Esta foto de Autor desconocido está bajo licencia CC BY

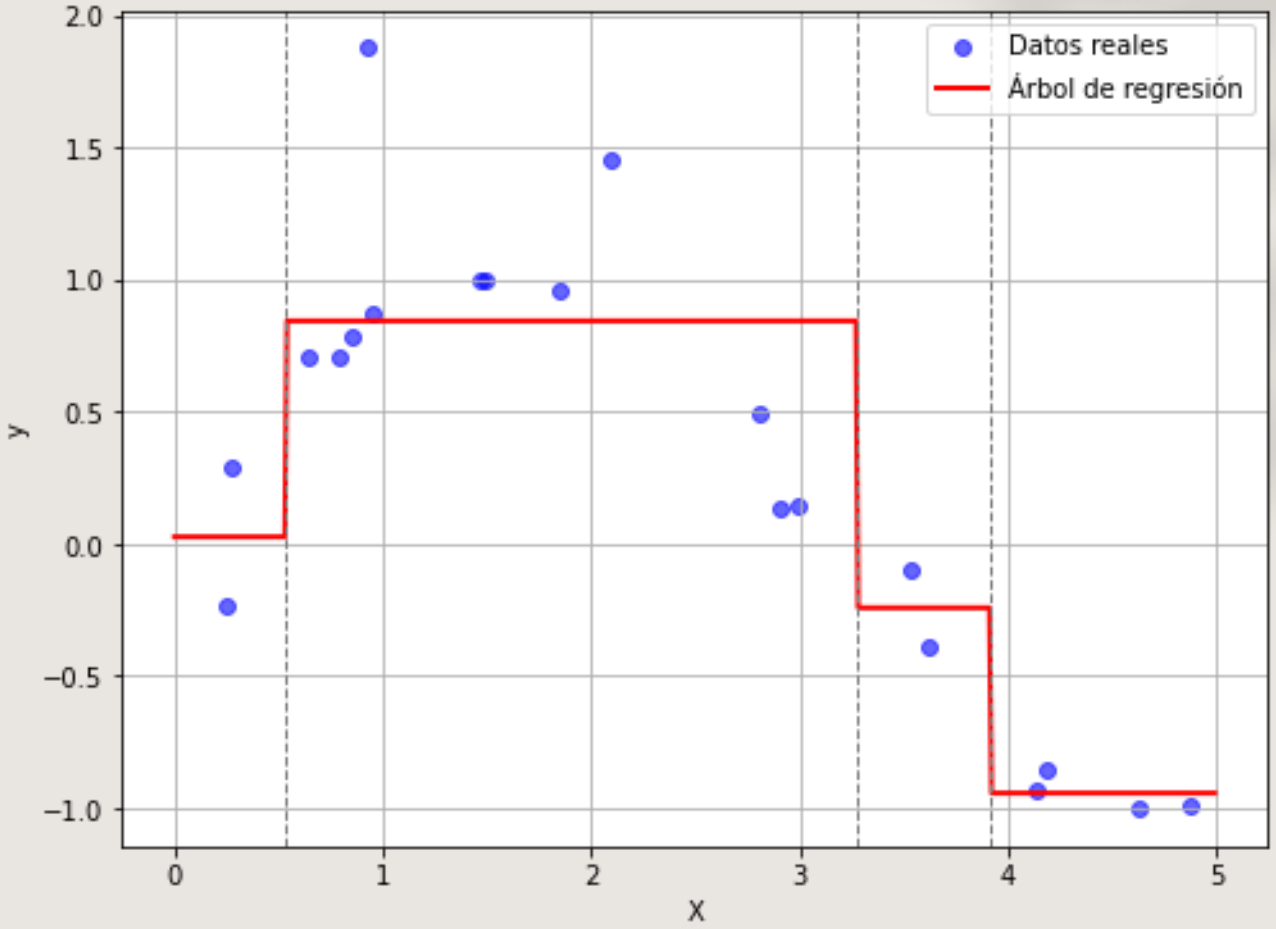
Por qué se ve así

Un árbol de regresión **divide el espacio en intervalos** (segmentos) según los umbrales que aprendió.

Dentro de cada intervalo, el árbol **predice el promedio** de los valores de y en ese segmento.

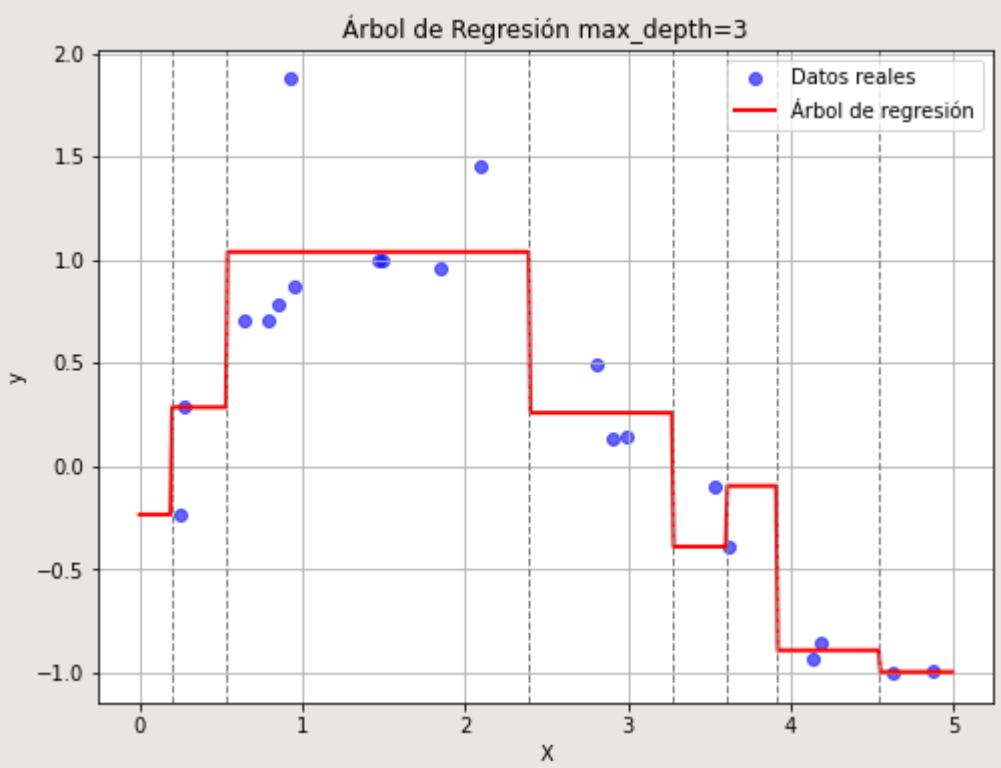
Por eso la línea roja no es suave como una regresión lineal, sino que tiene **tramos horizontales**:

- Cada tramo corresponde a una **hoja del árbol**.
- Cuando el árbol cambia de hoja, la línea salta a otro nivel.

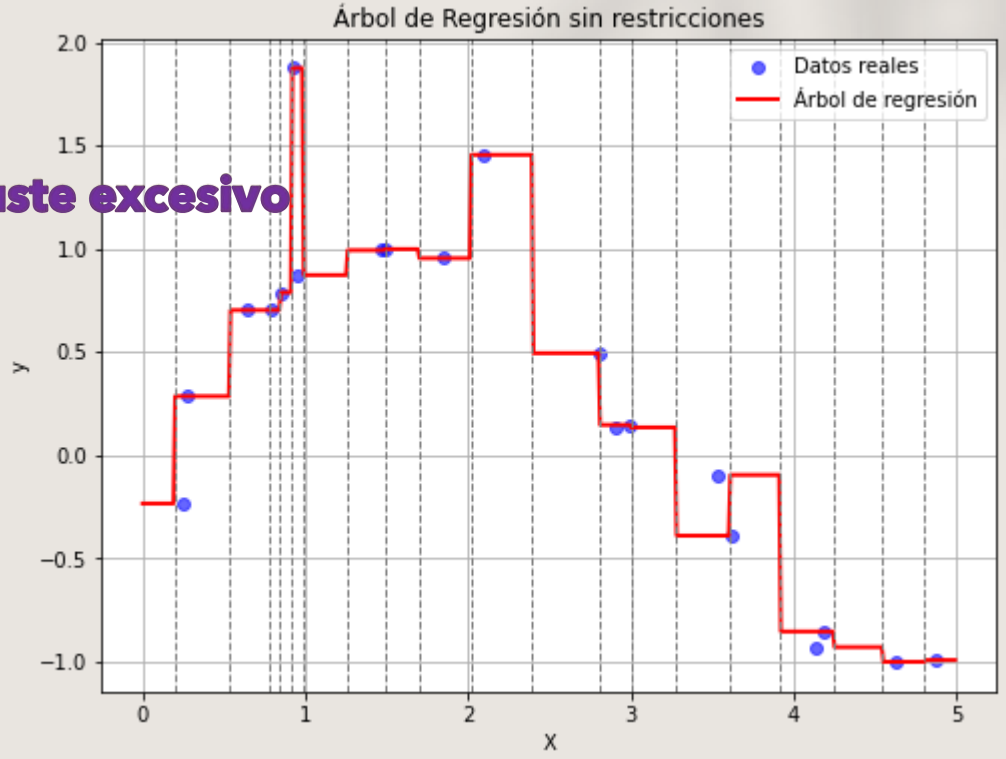


Función de coste para el algoritmo CART

$$J\left(k, t_k\right)=\frac{m_{\text{left}}}{m} \text{MSE}_{\text{left}}+\frac{m_{\text{right}}}{m} \text{MSE}_{\text{right}} \quad \text{where} \begin{cases}\text{MSE}_{\text{node}}=\sum_{i \in \text{node}}\left(\hat{y}_{\text{node}}-y^{(i)}\right)^2 \\ \hat{y}_{\text{node}}=\frac{1}{m_{\text{node}}} \sum_{i \in \text{node}} y^{(i)}\end{cases}$$



sobreajuste excesivo



Resumen del Dataset California Housing
Observaciones: 20,640.
Variables predictoras (todas numéricas continuas):

- MedInc: Ingreso medio (decenas de miles de dólares).
- HouseAge: Edad promedio de las casas.
- AveRooms: Promedio de habitaciones por hogar.
- AveBedrms: Promedio de dormitorios por hogar.
- Population: Población total del distrito.
- AveOccup: Promedio de ocupantes por hogar.
- Latitude: Latitud.
- Longitude: Longitud.

Variable objetivo: Precio medio de las casas (en cientos de miles de dólares).

El **squared error** es la suma de los cuadrados de las diferencias entre cada valor real y el promedio del nodo:

$$\text{squared error} = \sum_{i=1}^{n} (y_i - \bar{y})^2$$

donde:

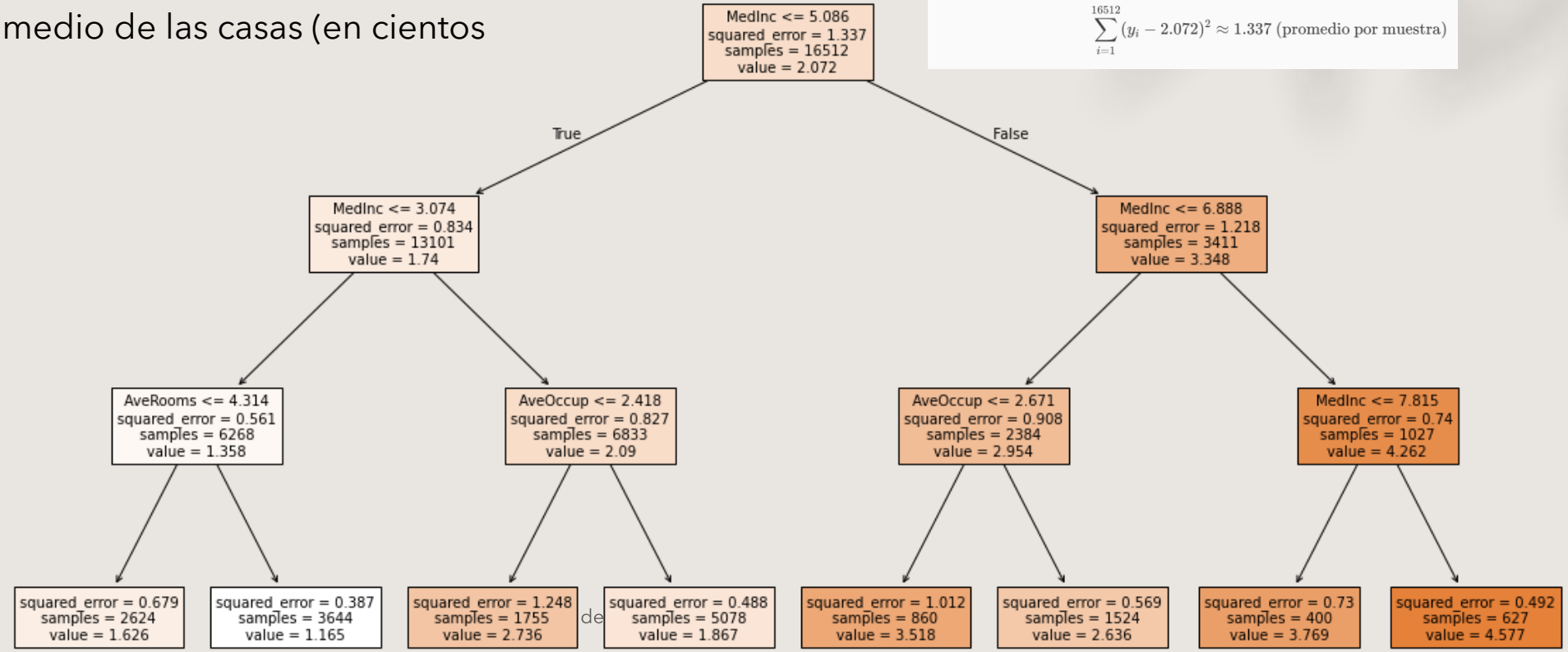
- y_i = valor real de la variable objetivo para la muestra i
- $\bar{y}$ = promedio de la variable objetivo en el nodo
- n = número de muestras en el nodo

- `samples = 16512`
- `value = 2.072`
- `squared error = 1.337`

Esto significa que, si calculamos:

$$\sum_{i=1}^{16512} (y_i - 2.072)^2 \approx 1.337 \text{ (promedio por muestra)}$$

Árbol de regresión (max_depth=4) para predicción de pr



Árboles de decisión

- Entrenamiento y visualización
- Efectos de los hiperparámetros en el desempeño del árbol
- Generación de predicciones



Tabla comparativa de algoritmos de árboles de decisión

Algoritmo	Criterio de división	Tipo de variables *input*	Estrategia de poda
ID3	Ganancia de información	Solo categóricas	No poda
CART	Índice de Gini	Categóricas y numéricas	Poda basada en el coste de complejidad
C4.5	Ratio de ganancia	Categóricas y numéricas	Poda basada en el error



¿Cuál es la profundidad aproximada de un árbol de decisión entrenado (sin restricciones) en un conjunto de entrenamiento con un millón de observaciones?

Profundidad

- La profundidad aproximada de un árbol de decisión entrenado sin restricciones (es decir, sin límite en max_depth, min_samples_split, etc.) depende de cuántas divisiones binarias se hacen antes de llegar a las hojas donde hay solo una observación.

$$2^d \cong N$$

$$d = \log_2 N,$$

La profundidad aproximada de un árbol de decisión entrenado sin restricciones en un conjunto de un millón de observaciones es de **alrededor de 20 niveles**.

¿Cómo influye el número de observaciones en la profundidad de un árbol sin restricciones?

a) No influye en la profundidad
b) A mayor número de observaciones, menor profundidad
c) A mayor número de observaciones, mayor profundidad
d) Depende solo del número de variables

¿Cómo influye el número de observaciones en la profundidad de un árbol sin restricciones?

a) No influye en la profundidad
b) A mayor número de observaciones, menor profundidad
c) A mayor número de observaciones, mayor profundidad ✓
d) Depende solo del número de variables

¿Cómo influye el número de variables en la construcción del árbol?

a) Más variables siempre implican mayor profundidad
b) Más variables ofrecen más opciones de división
c) Menos variables hacen el árbol más ancho
d) No afecta la estructura del árbol

¿Cómo influye el número de variables en la construcción del árbol?

a) Más variables siempre implican mayor profundidad
b) Más variables ofrecen más opciones de división ✓
c) Menos variables hacen el árbol más ancho
d) No afecta la estructura del árbol

Si dos variables están altamente correlacionadas, ¿qué puede ocurrir?

- a) El árbol nunca las usa
- b) El árbol elige una y descarta la otra
- c) El árbol las usa siempre en la raíz
- d) No afecta la selección de variables

Si dos variables están altamente correlacionadas, ¿qué puede ocurrir?

- a) El árbol nunca las usa
- b) El árbol elige una y descarta la otra ✓
- c) El árbol las usa siempre en la raíz
- d) No afecta la selección de variables

¿Qué pasa si agregamos variables irrelevantes?

- a) El árbol se simplifica
- b) El árbol puede volverse más complejo
- c) El árbol mejora su precisión
- d) No cambia nada

¿Qué pasa si agregamos variables irrelevantes?

- a) El árbol se simplifica
- b) El árbol puede volverse más complejo ✓
- c) El árbol mejora su precisión
- d) No cambia nada

¿Cómo se determina qué variable se elige en la raíz del árbol?

- a) Al azar
  b) Por la que tiene más valores únicos
  c) Por la que produce máxima ganancia de información o máxima reducción de impureza.
  d) Por la que aparece primero en el dataset

¿Cómo se determina qué variable se elige en la raíz del árbol?

- a) Al azar
  b) Por la que tiene más valores únicos
  c) Por la que produce máxima ganancia de información o máxima reducción de impureza.✓
  d) Por la que aparece primero en el dataset

¿Puede una misma variable aparecer en varios niveles del árbol?

- a) No, nunca
- b) Sí, si sigue siendo útil para separar clases
- c) Solo si hay pocas variables
- d) Solo en árboles podados

¿Puede una misma variable aparecer en varios niveles del árbol?

- a) No, nunca
- b) Sí, si sigue siendo útil para separar clases ✓
- c) Solo si hay pocas variables
- d) Solo en árboles podados



¿La impureza de Gini de un nodo suele ser menor o mayor que la de su nodo padre?

Si un árbol de decisión está sobreajustando el conjunto de entrenamiento, ¿es recomendable disminuir la profundidad máxima (max_depth)?

Si un árbol de decisión está subajustando el conjunto de entrenamiento, ¿es recomendable escalar las características de entrada?

¡Pausa para un cafecito!





Esta foto de Autor desconocido está bajo licencia CC BY-SA-NC

Ensemble Methods

Aprendizaje por Conjuntos





🌲 🌲 🌲 Introducción a *Random Forest*

- Es un algoritmo de aprendizaje supervisado que se utiliza tanto para **clasificación** como para **regresión**.
- Fue desarrollado por Leo Breiman y Adele Cutler.
- Se basa en la idea de construir **múltiples árboles** de decisión y combinar sus resultados para mejorar la precisión y evitar el sobreajuste.

🔍 ¿Cómo funciona?

1. **Bootstrap:** Se generan múltiples subconjuntos de datos (con reemplazo) a partir del conjunto de entrenamiento original.
2. **Árboles de decisión**: Para cada subconjunto, se construye un árbol de decisión. Pero en cada nodo, en lugar de considerar todas las variables, se selecciona aleatoriamente un subconjunto de ellas.
3. **Votación o promedio**: En clasificación, cada árbol vota por una clase y se elige la clase con más votos. En regresión, se promedia la predicción de todos los árboles.

🌲 🌲 🌲 Introducción a *Random Forest*

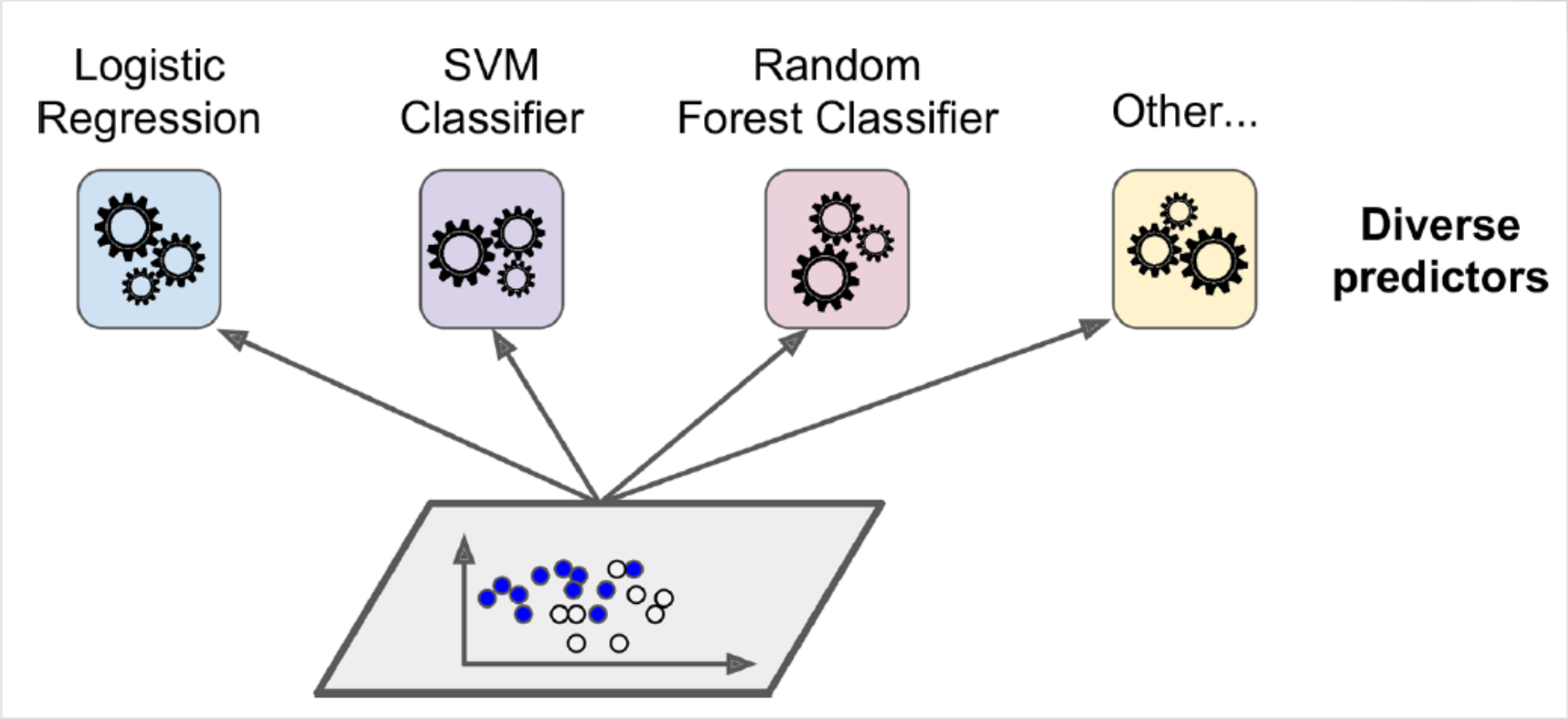
🎯 Ventajas

- **Robusto al sobreajuste**: Al promediar muchos árboles, se reduce la varianza.
- **Manejo de datos faltantes y ruido**.
- **Importancia de variables**: Permite evaluar qué variables son más relevantes para la predicción.
- **Escalable**: Funciona bien con grandes volúmenes de datos.

⚠️ Desventajas

- **Menos interpretabilidad** que un solo árbol de decisión.
- **Más costoso computacionalmente**.
- Puede ser sensible a clases desbalanceadas si no se ajusta correctamente.

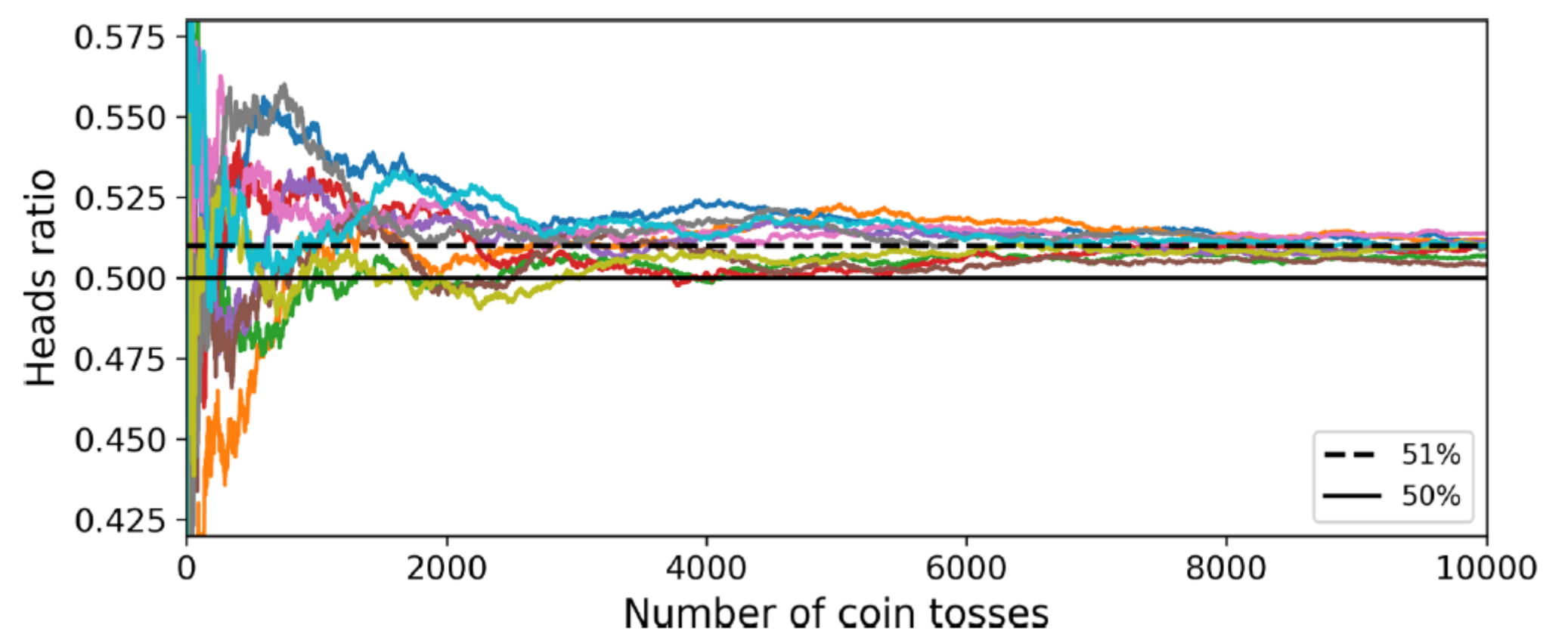
Clasificadores por Votación



¡Por qué funciona?

Analogía de la moneda sesgada
Moneda con **51% probabilidad de cara** y **49% cruz**.
Si lanzas **1.000 veces**, obtendrás ≈ **510 caras vs 490 cruces** → mayoría de caras.
Probabilidad de mayoría de caras:

1.000 lanzamientos → ~75%
10.000 lanzamientos → >97%

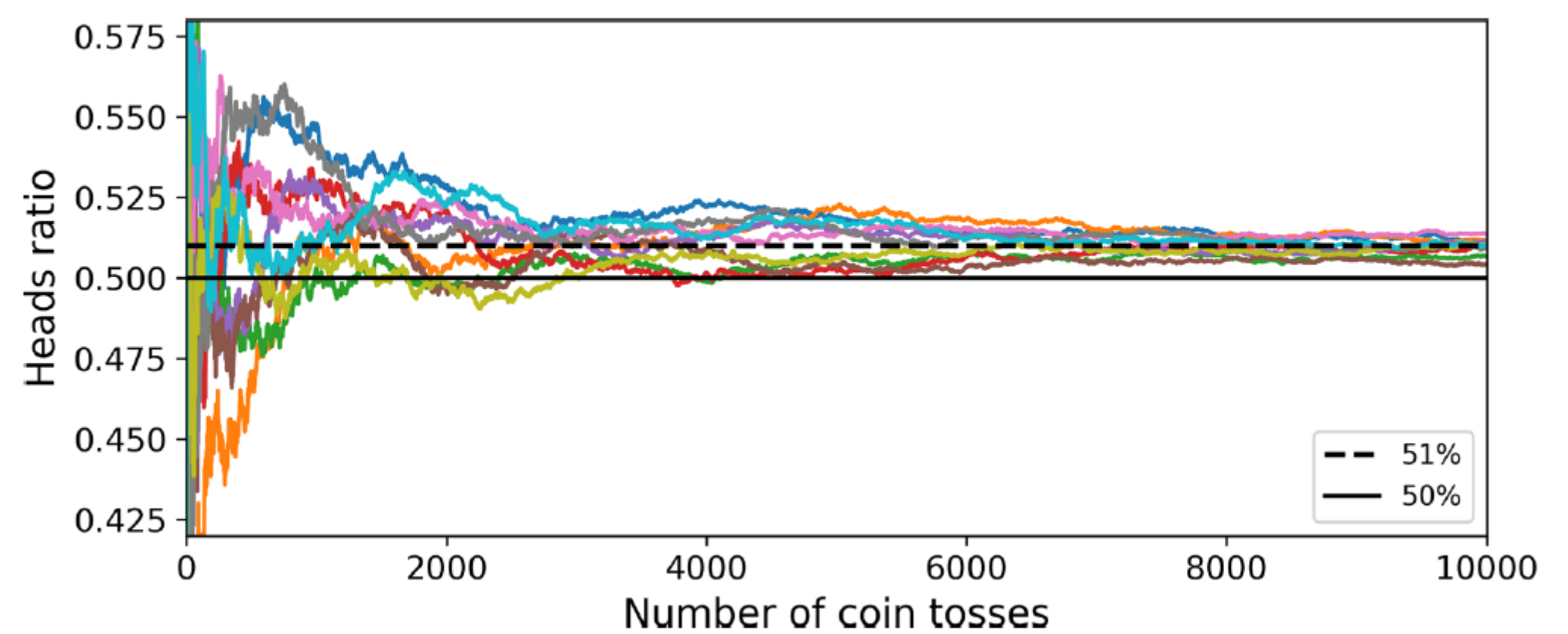


¿Por qué funciona?

Ley de los Grandes Números

A medida que aumentan los lanzamientos:

- La proporción de caras → **se aproxima al 51% real**.
- Las series terminan **consistentemente por encima del 50%**.



Relación con Voting Classifier

- Cada clasificador débil = **moneda ligeramente sesgada**.
- Combinando muchos clasificadores:
  - **Más votos → mayor precisión global**.
- Incluso si cada modelo es débil, el conjunto puede ser **un aprendiz fuerte**.



Voting Classifier: Definición y Ventajas

´¿Qué es?

- Combina predicciones de varios clasificadores.
- Predice la clase con mayoría de votos → Hard Voting.
- Variante: Soft Voting (usa probabilidades promedio).

Ventajas

- **Mejor precisión** que el mejor clasificador individual.
- Funciona incluso con **aprendices débiles**, si son:
  - **Suficientemente numerosos**.
  - **Diversos** (no todos cometen los mismos errores).



¿Cuál es el problema que tienen los árboles de clasificación o los de regresión?

MODELO Sobreajuste

Entrenamiento **Test** VARIANZA



SESGO





Árboles de Decisión y el Problema

- **Compromiso Sesgo-Varianza**

  Sesgo bajo: se ajustan muy bien a los datos de entrenamiento.

  Varianza alta: sensibles a cambios en los datos → árboles muy diferentes.

- **Consecuencia**

  Buen desempeño en entrenamiento, pero baja capacidad de generalización.

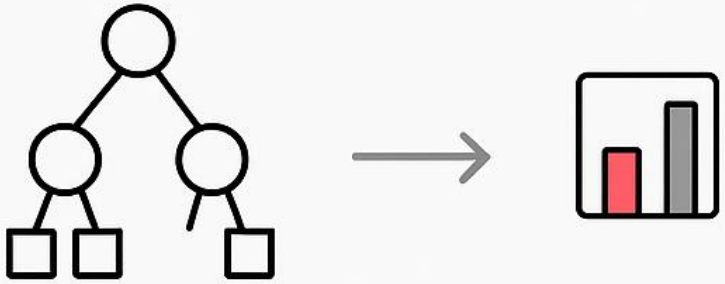
  Predicciones inestables con datos nuevos.

Solución – Árboles de Precisión (Ensemble)

- **Objetivo:** mantener sesgo bajo y reducir varianza.
- **Cómo se logra**:
  - Entrenar varios árboles en lugar de uno.
  - Usar subconjuntos aleatorios de datos para cada árbol.
  - Agregar resultados (votación o promedio) para la predicción final.
- **Ventaja:** árboles menos precisos tienen menor impacto → modelo más robusto.

Árboles de Decisión y el Problema

- **Compromiso Sesgo-Varianza**
  - Sesgo bajo: se ajustan muy bien a los datos de entrenamiento
  - Varíanza alta: sensibles a cambios en los datos → arboles muy diferentes
- **Consecuencia**
  - Buen desempeño en entrenamíento, pero baja capacidad de generalización
  - Predicciones inesttables con datos nuevos

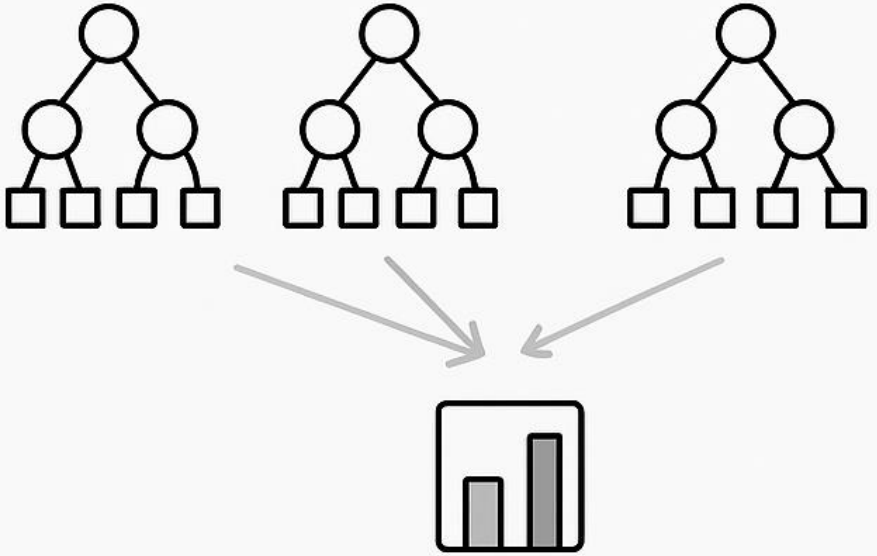


Solución – Árboles de Precisión (Ensemble)

Objetivo: mantener sego bajo y reducir varíanza

Cómo se logra:

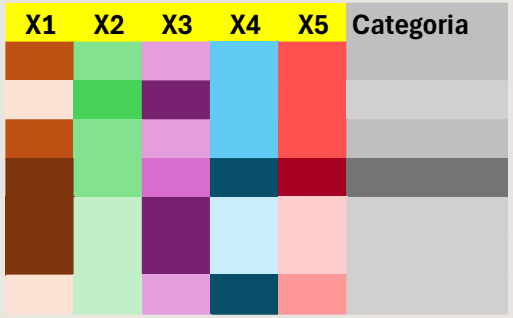
1. Entrenar varíos árboles en lugar de uno
2. Usar subconjuntos aleatorios de datos para cada arbol
3. Agregar resultados (votación o prommélío) para la predicciŏn final

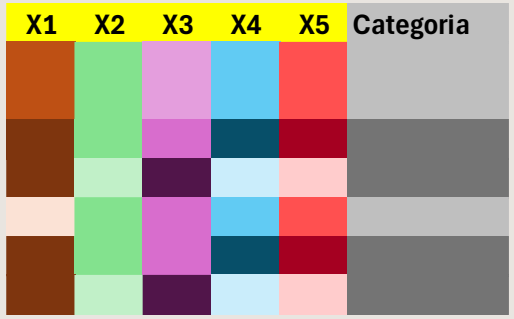


¿Cómo funcionan?



Muestreo con reposición.



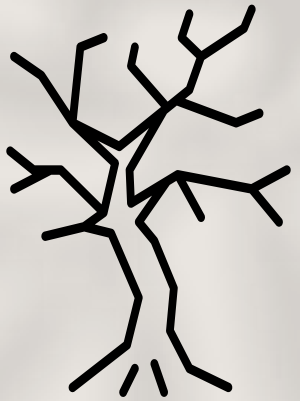


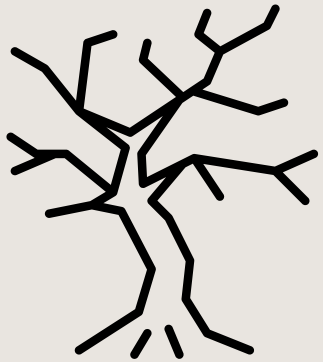
En el árbol entran algunas variables.





Se entrenan modelos. Árboles simples.





Clasificación



Se escoge la categoría asignada por la mayoría de los árboles.

Regresión



Se promedia la predicción hecha por los árboles.

Pasos

1. Bootstrapping en Random Forest

- Se parte del conjunto de datos original con n observaciones.
- Para cada árbol del bosque: Se toma una **muestra bootstrap** del conjunto original (mismo tamaño n, con reemplazo).
- Esto significa que algunas observaciones se repiten y otras quedan fuera (estas últimas se llaman **out-of-bag**).

2. Entrenamiento de cada árbol

- Cada árbol se entrena con su muestra bootstrap.
- Además, en cada división del árbol, **se selecciona aleatoriamente un subconjunto** de variables (**feature bagging**).
- Esto **introduce variabilidad** y **reduce la correlación** entre árboles.

Pasos

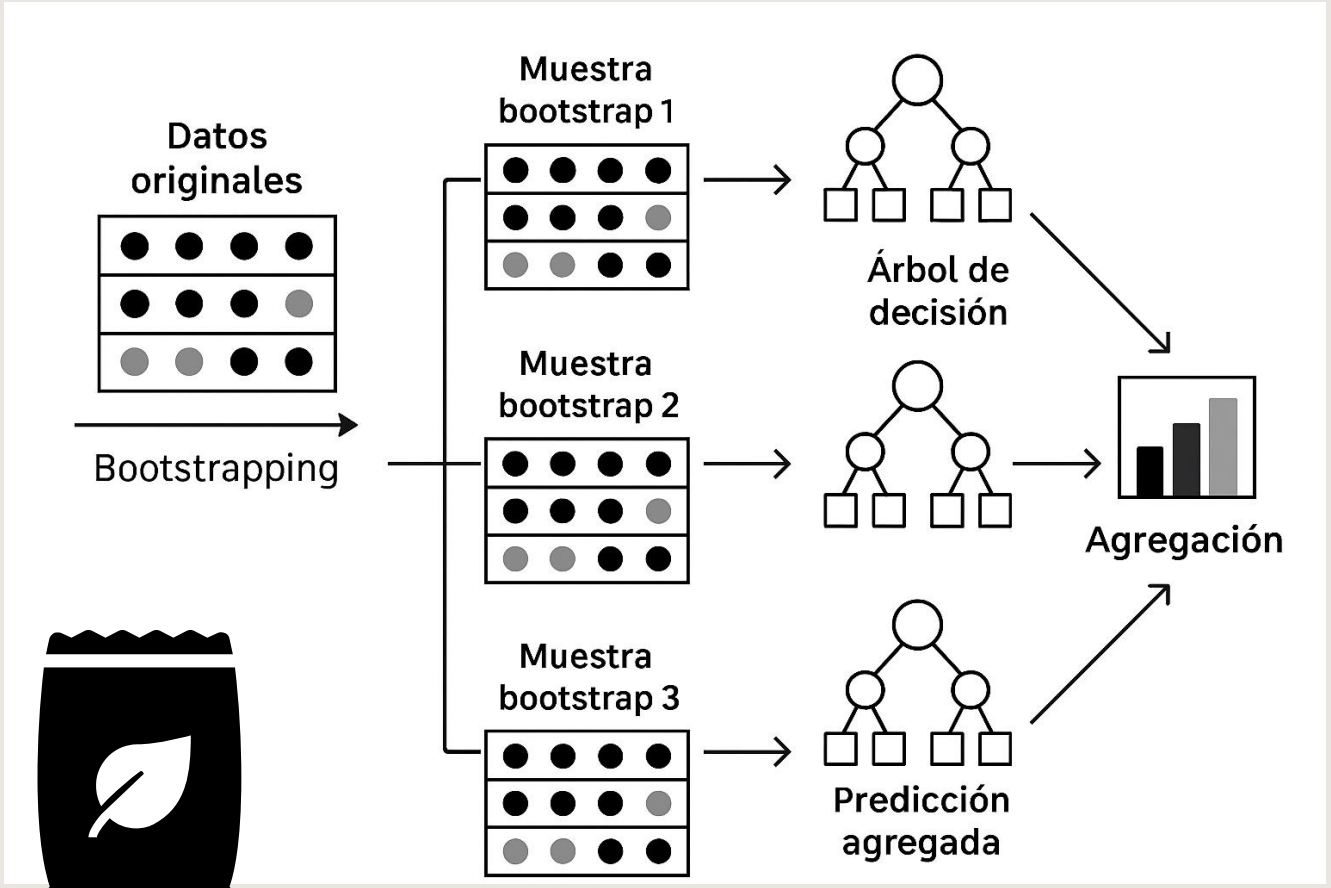
3. Predicción y agregación

- Cada árbol produce una predicción.
- El modelo final combina todas las predicciones:
  - **Clasificación:** votación mayoritaria.
  - **Regresión:** promedio.

¿Por qué es importante el bootstrapping?

- Permite que cada árbol vea una versión distinta del dataset → **reduce varianza**.
- Genera diversidad entre árboles, lo que mejora la capacidad de generalización.
- Las observaciones **out-of-bag** se usan para estimar el error sin necesidad de un conjunto de validación adicional.

Diagrama del proceso de **bootstrapping** en Random Forest:



- **Datos originales** → se generan varias **muestras bootstrap** (con reemplazo).
- Cada muestra entrena un **árbol de decisión**.
- Las predicciones de todos los árboles se **agregan** (votación o promedio) para obtener la predicción final.

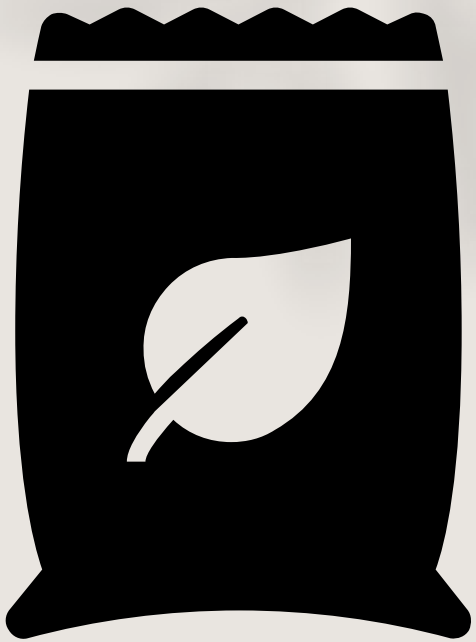
out-of-bag

Hiperparámetros



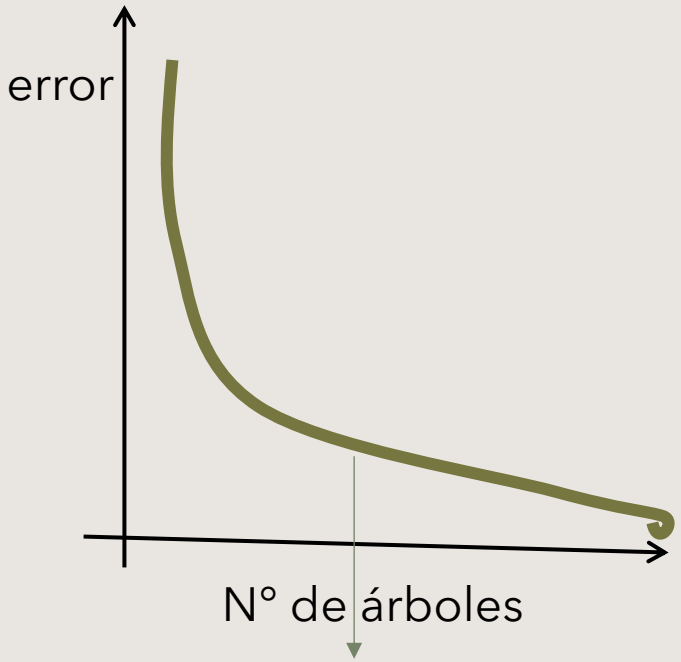
- **¿Cuántos árboles usaron?**
- **¿Cuántas características seleccionar?**
- **¿Hasta dónde hacemos crecer los árboles?**

out-of-bag

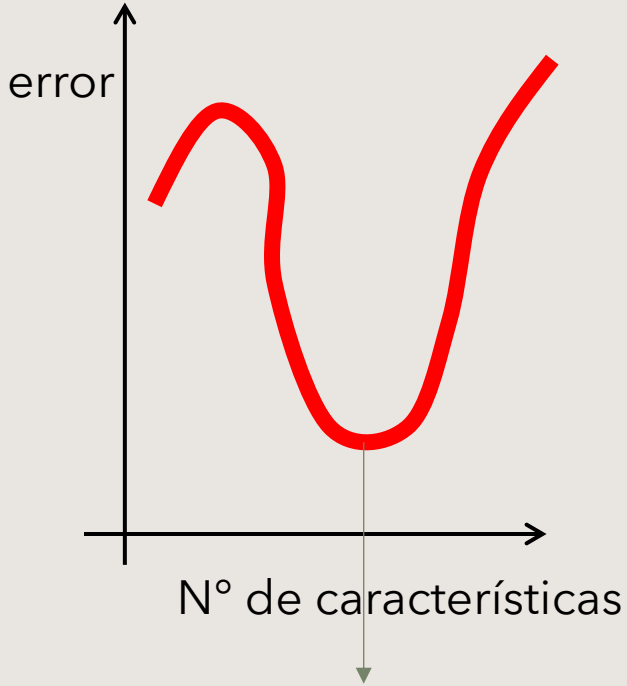


¿Cómo se comportan estos hiper parámetros?

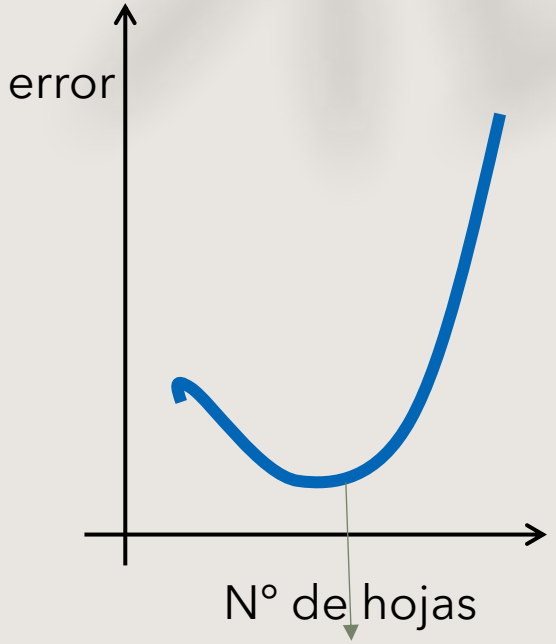
N° de arboles



N° de características



N° total de hojas



En Random Forest, las métricas para evaluar la calidad del modelo dependen del tipo de problema:

Clasificación

- Exactitud (Accuracy).
- Matriz de Confusión.
- Precisión, Recall y F1-Score.
- Error Out-of-Bag (OOB)Estimación interna del error usando las observaciones no incluidas en cada bootstrap.

Regresión

- Error Cuadrático Medio
- Raíz del Error Cuadrático Medio
- Coeficiente de Determinación (R^2): Mide qué proporción de la variabilidad se explica por el modelo.
- Error Out-of-Bag (OOB)También aplicable en regresión como estimación interna.

Bagging = Boostraping + Agregation

https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomTreesEmbedding.html

